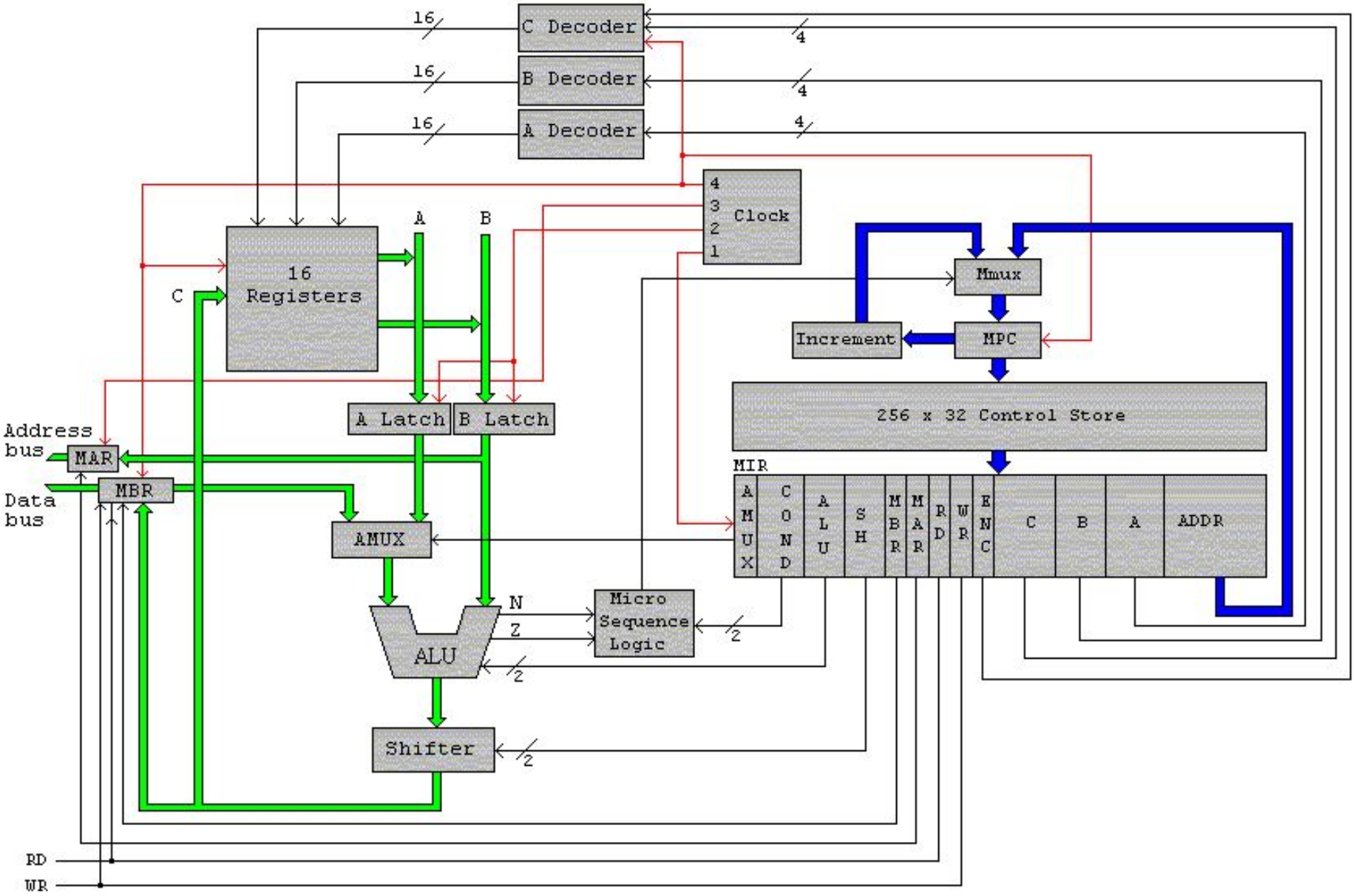
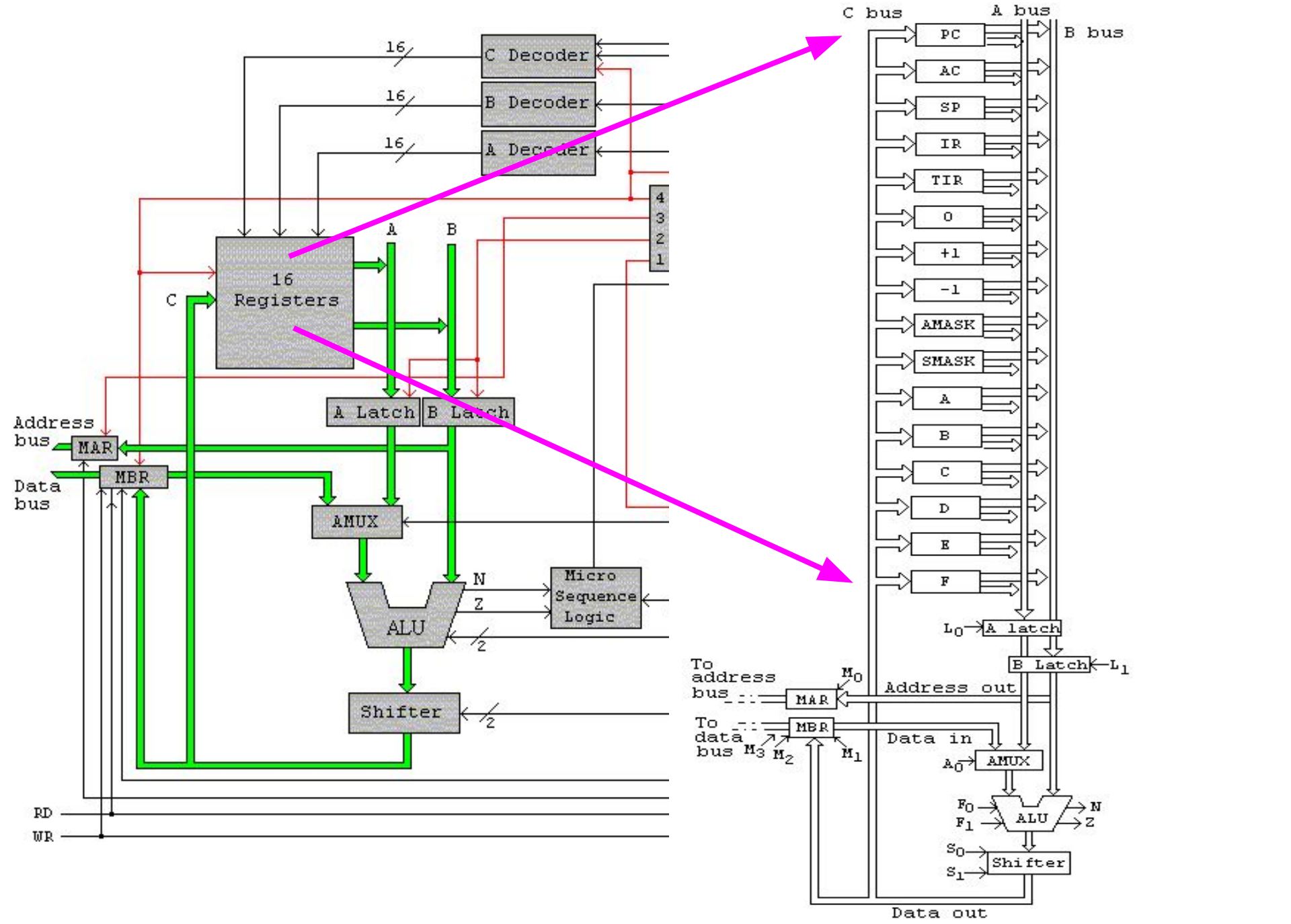
Arquitectura Horizontal

T1 - Trayectoria de ejecución

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta


C Decoder
B Decoder
A Decoder
16
4
Clock
16 Registers
A
B
C
Mmux
Increment
MPC
256 x 32 Control Store
MIR
A Latch
B Latch
Address bus
MAR
Data bus
MBR
AMUX
ALU
Shifter
N
Z
Micro Sequence Logic
AMUX
COND
ALU
SH
MBR
MAR
RD
WR
ENC
C
B
A
ADDR
RD
WR


C bus
A bus
B bus
PC
AC
SP
IR
TIR
0
+1
-1
AMASK
SMASK
A
B
C
D
E
F
16
C Decoder
B Decoder
A Decoder
4
3
2
1
16 Registers
A
B
C
A Latch
B Latch
Address bus
MAR
Data bus
MBR
AMUX
ALU
N
Z
Micro Sequence Logic
Shifter
RD
WR
L_0
A latch
B Latch
L_1
To address bus
M_0
Address out
To data bus
M_3
M_2
M_1
Data in
A_0
F_0
F_1
S_0
S_1
Data out

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<-y<-255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<-y<-255)

Conjunto de mnemónicos usados para programar la aplicación

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

Conjunto de mnemónicos usados para programar la aplicación

15 instr. con operando de 12 bits

6 instrucciones sin operando

2 instrucciones con operando de 8 bits

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

m[x] := ac

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

. . .

STOD *9*

. . .

Guardar el valor del AC en la memoria(9)

m[x] := ac

0001 ***0000 0000 1001***

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr

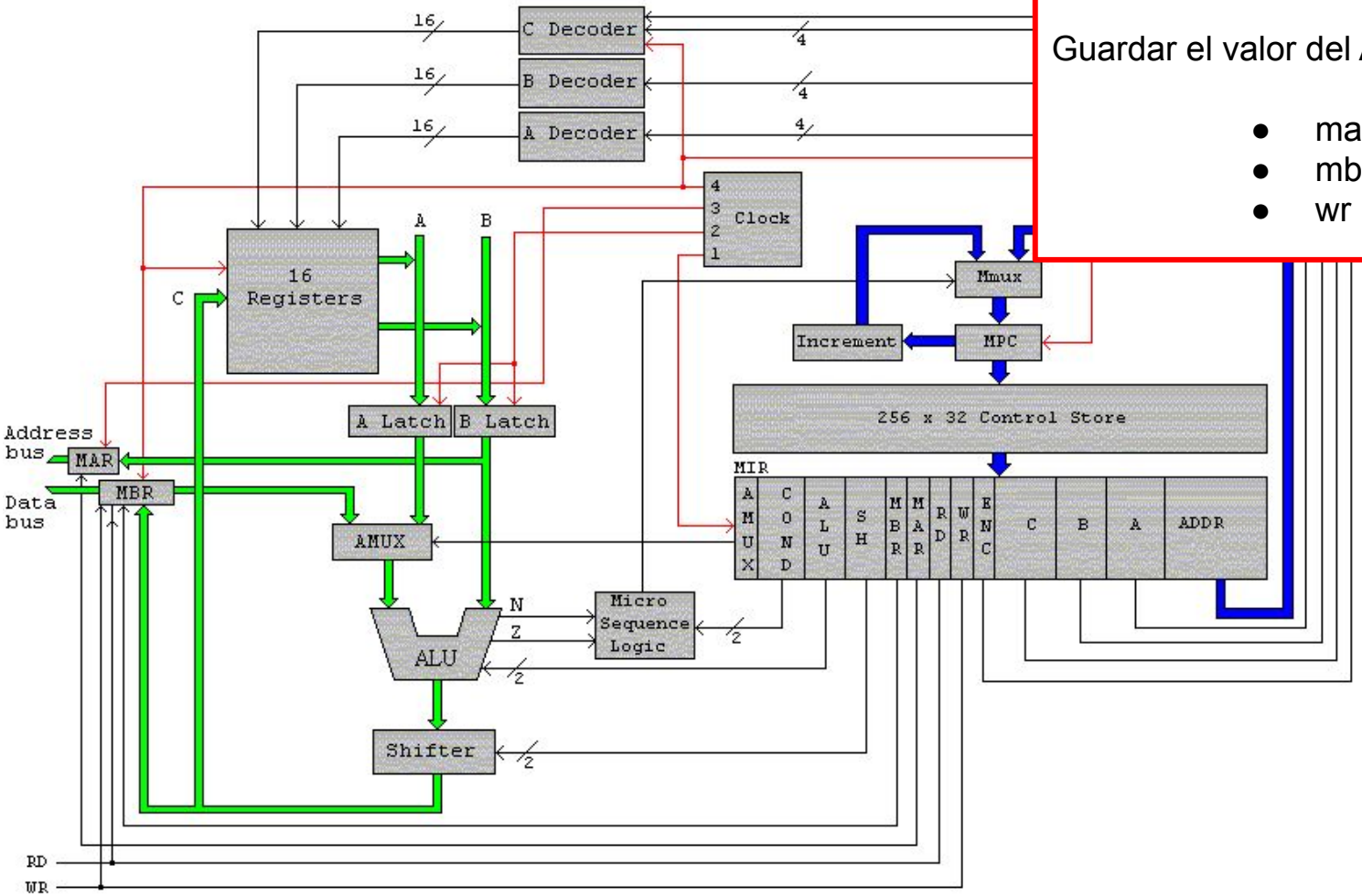
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



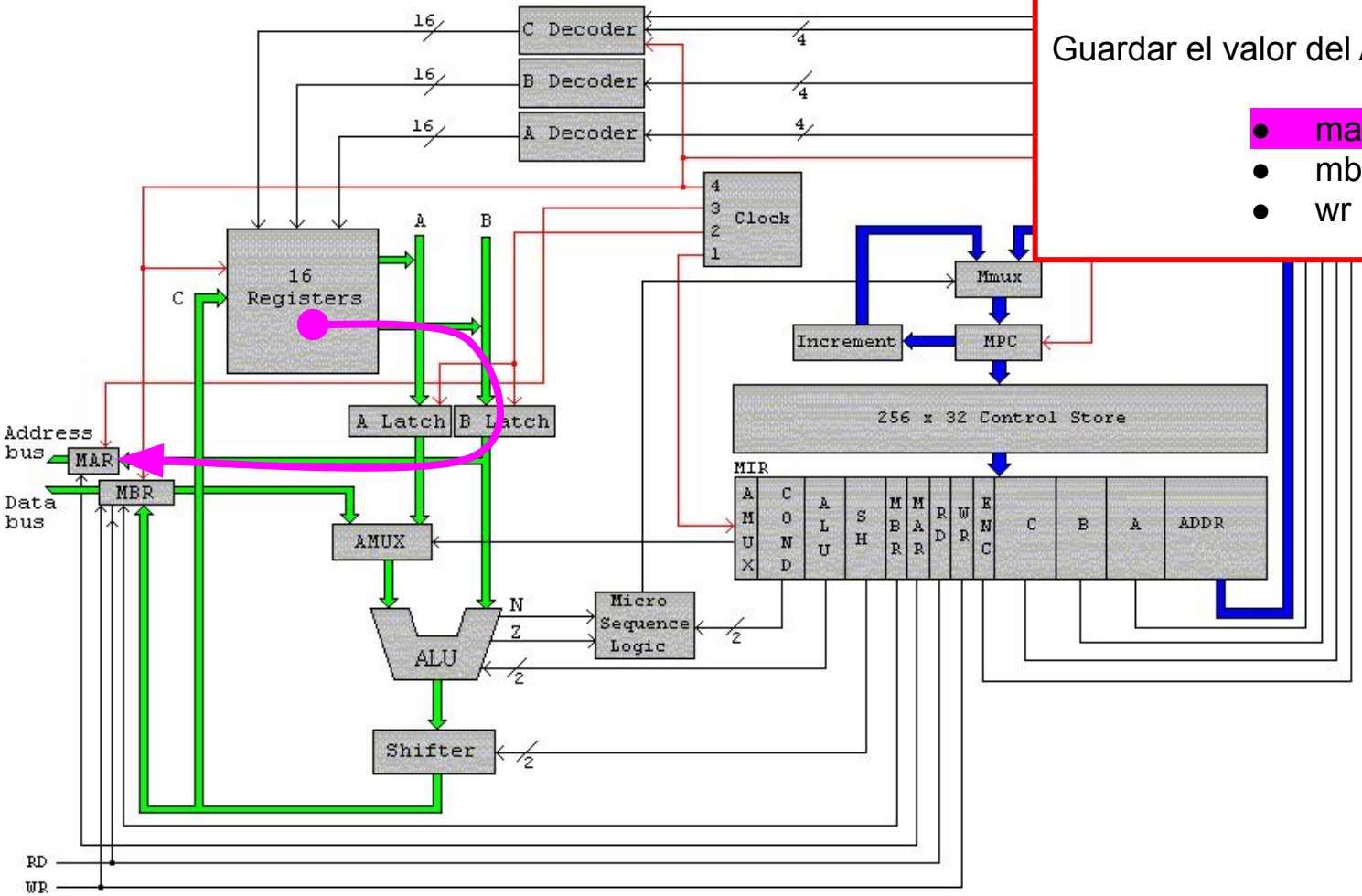
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



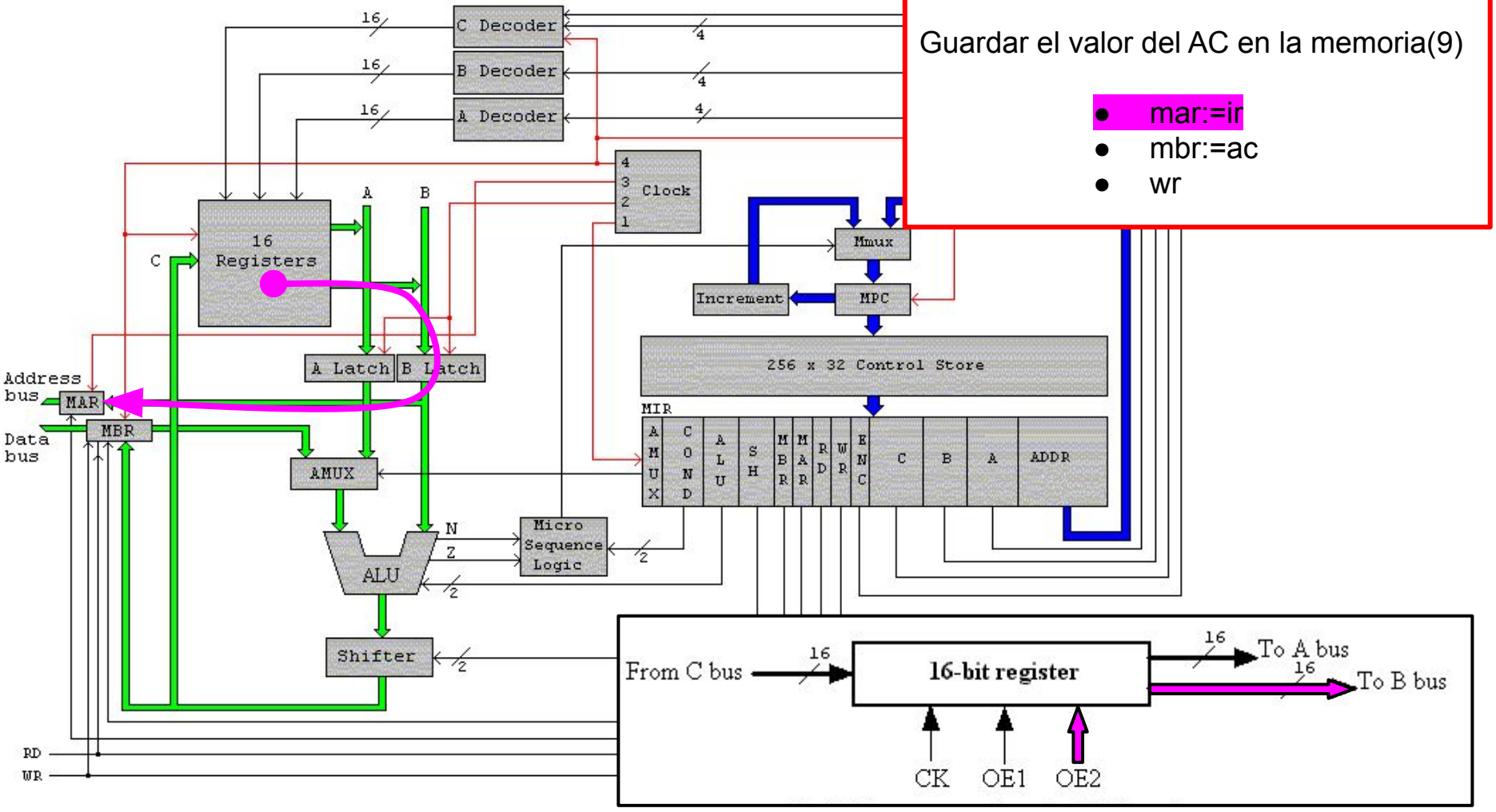
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



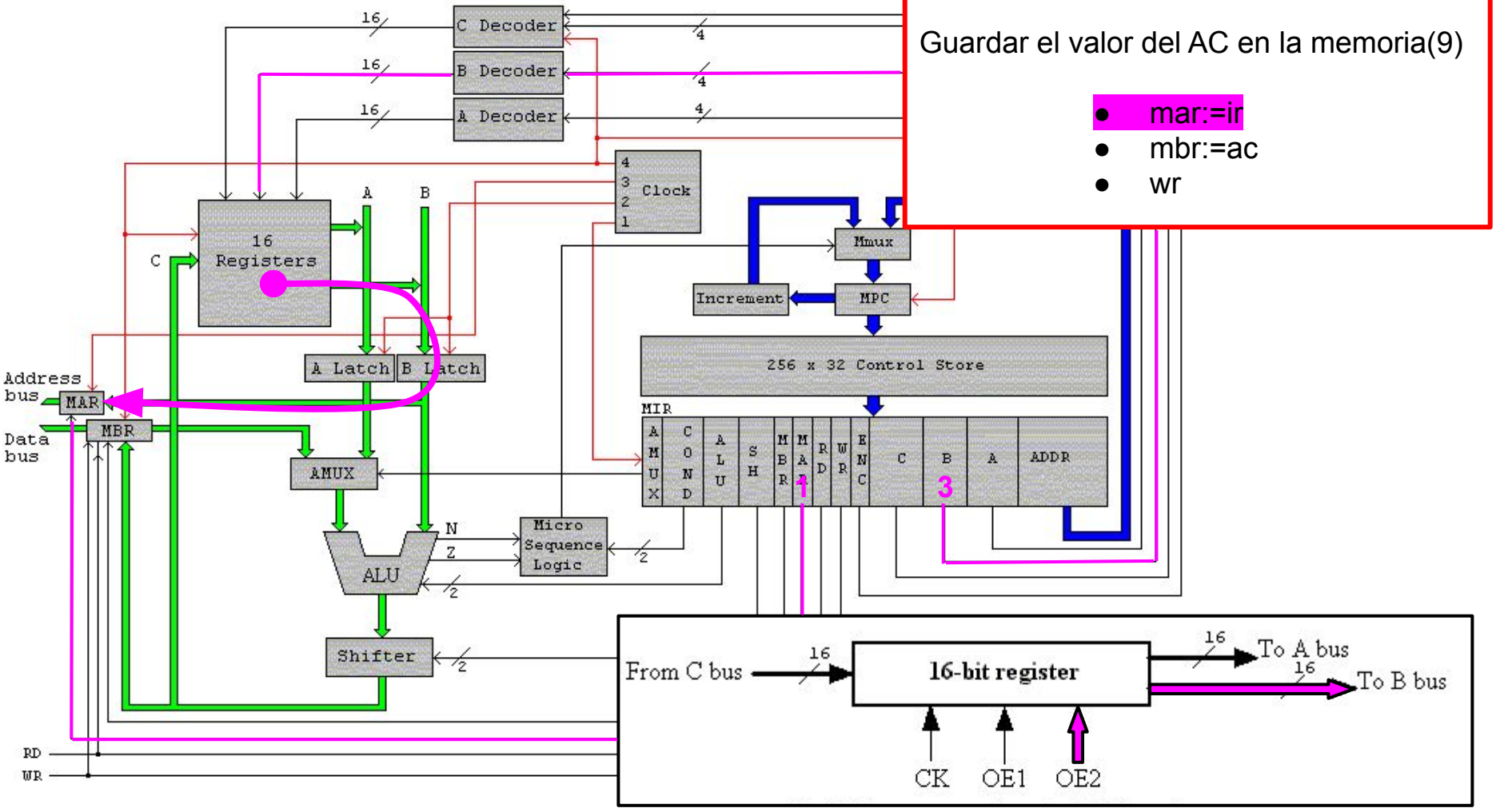
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



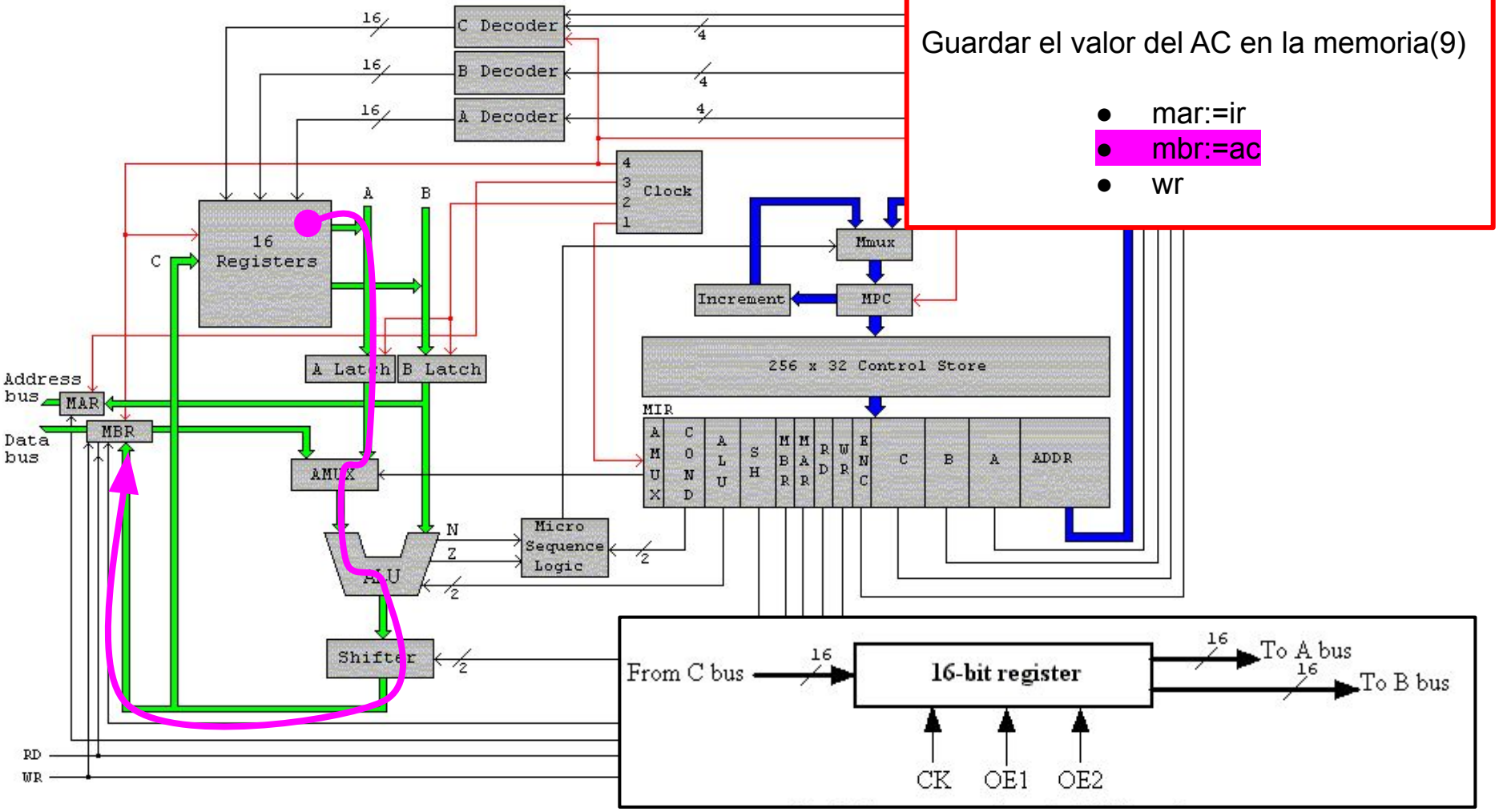
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



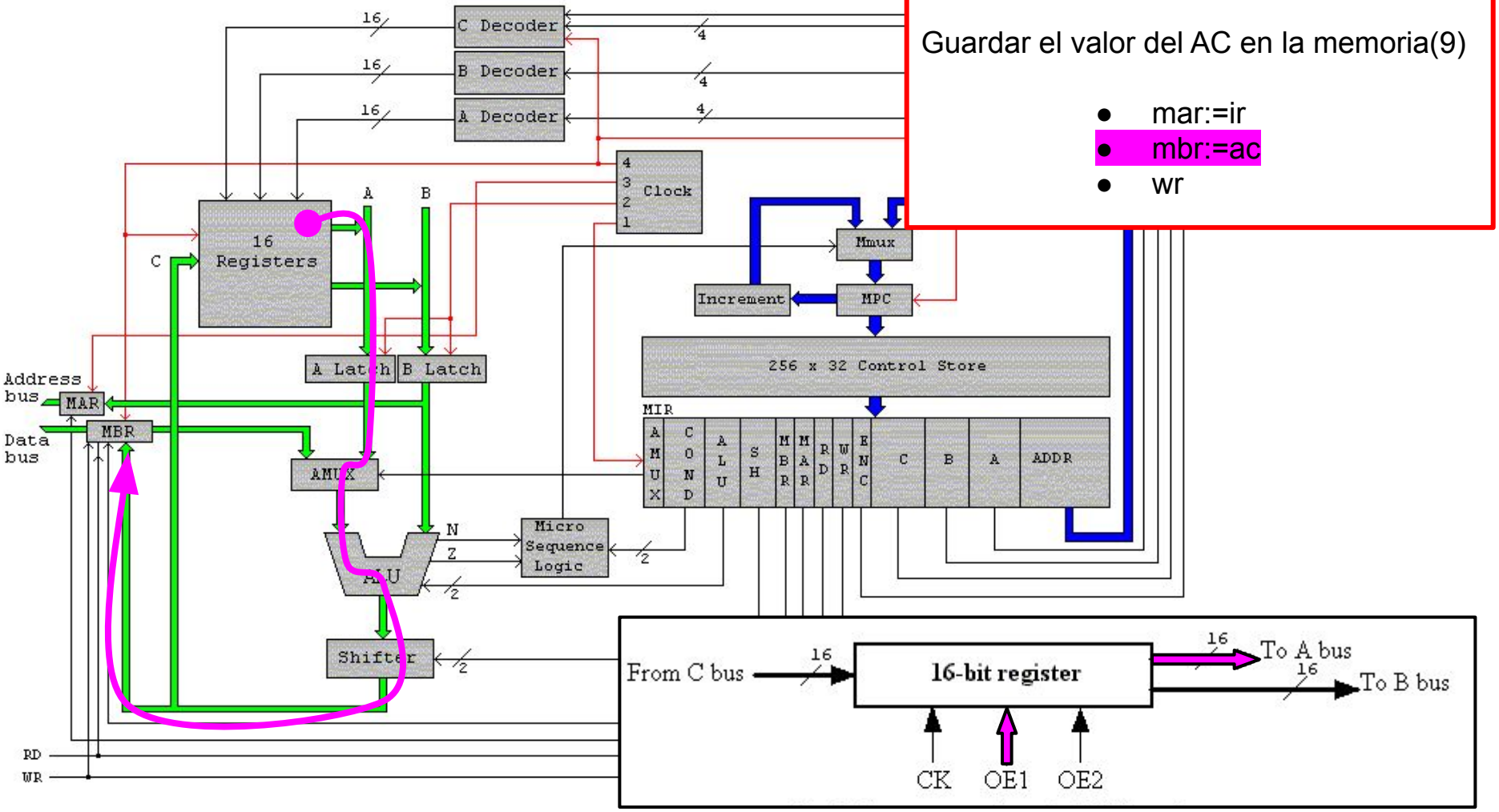
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



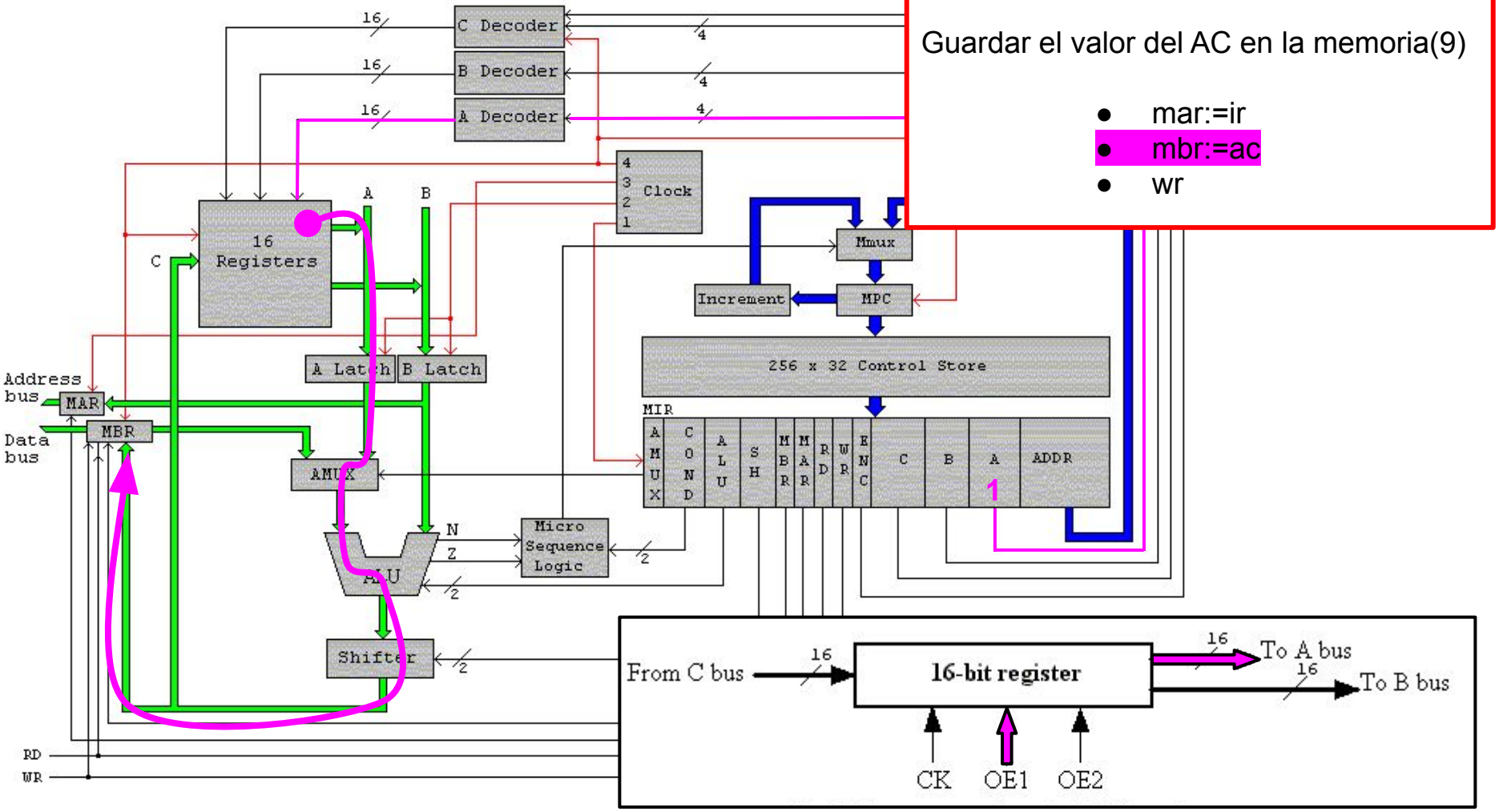
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



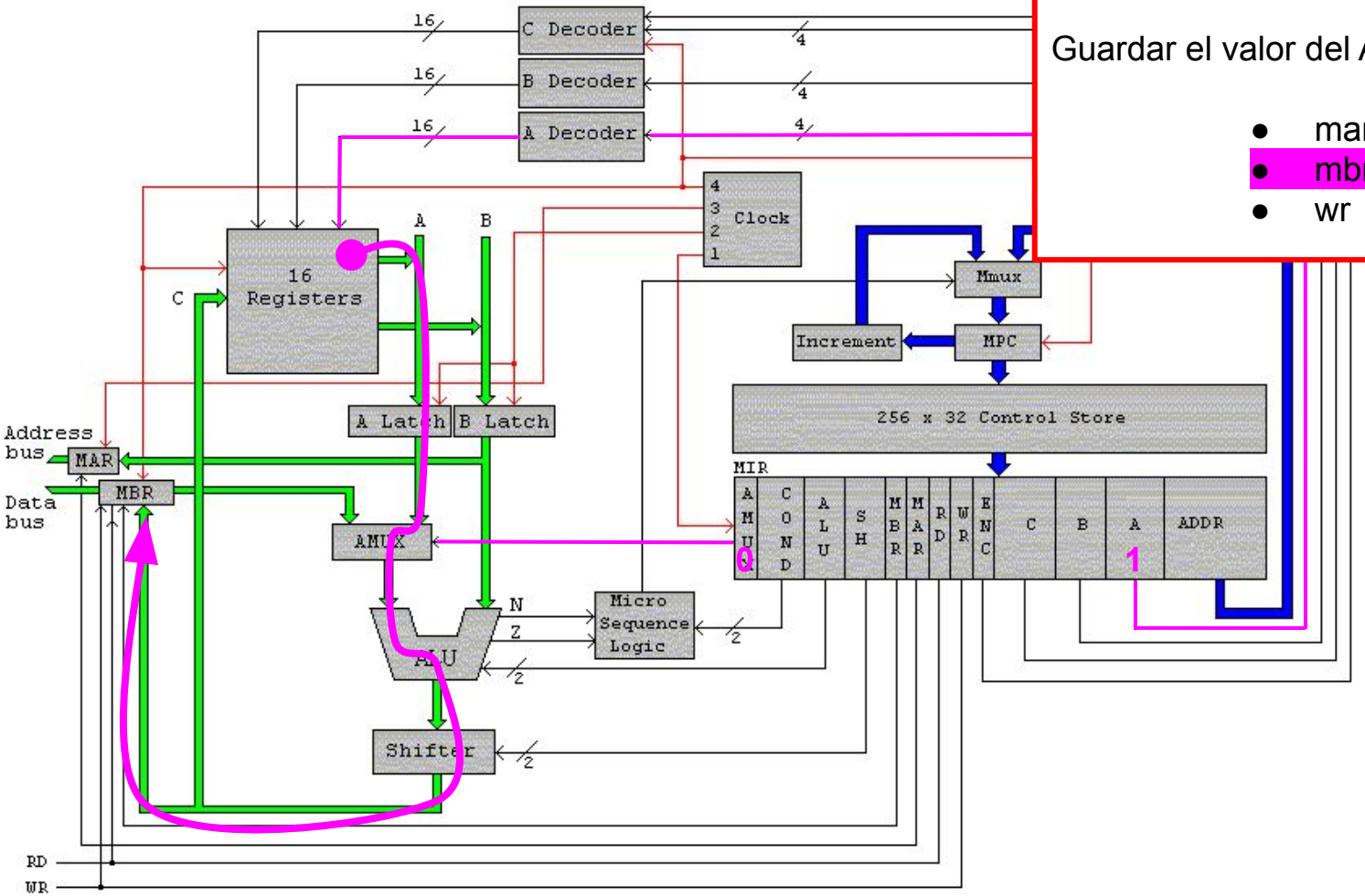
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



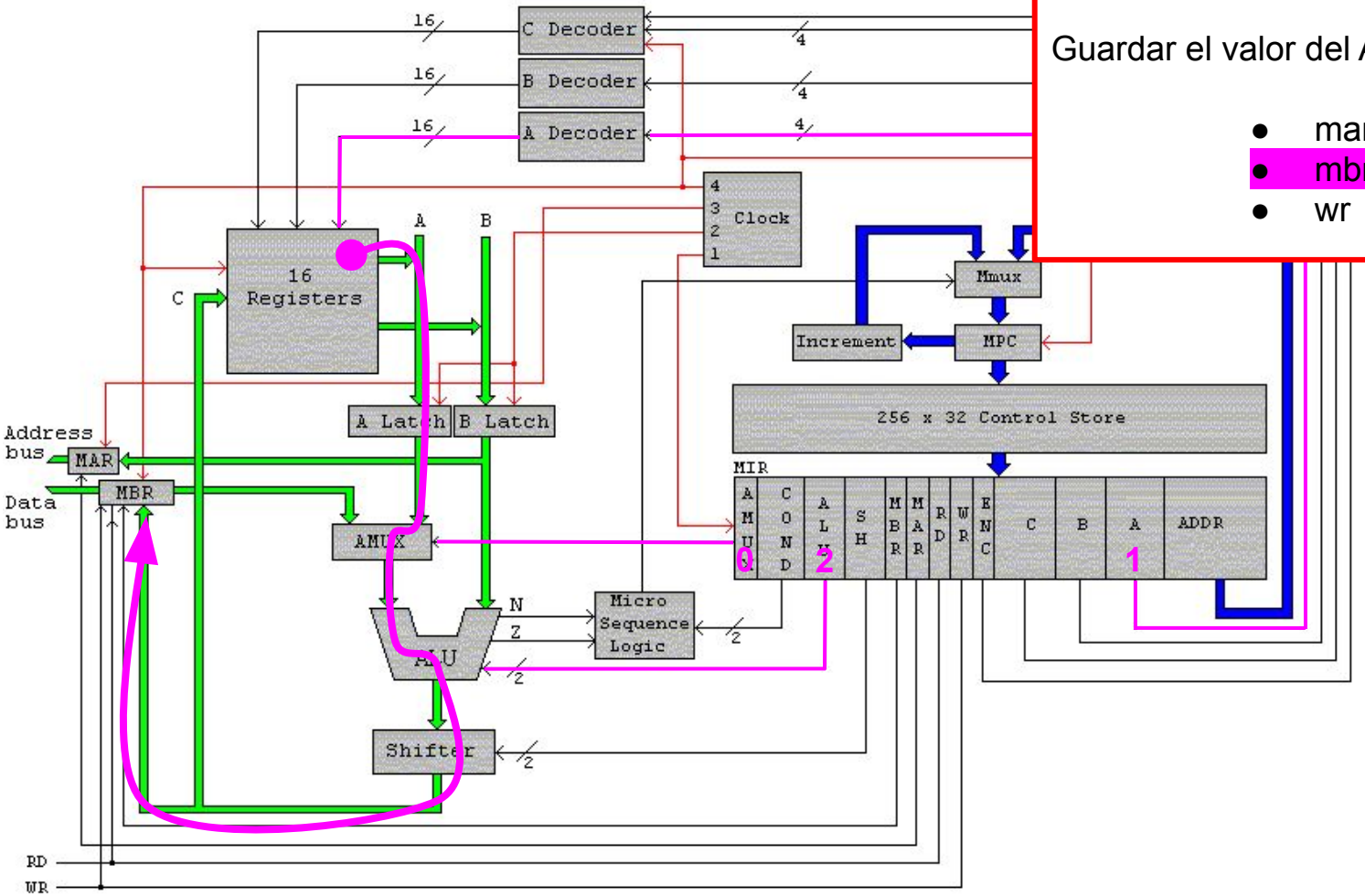
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



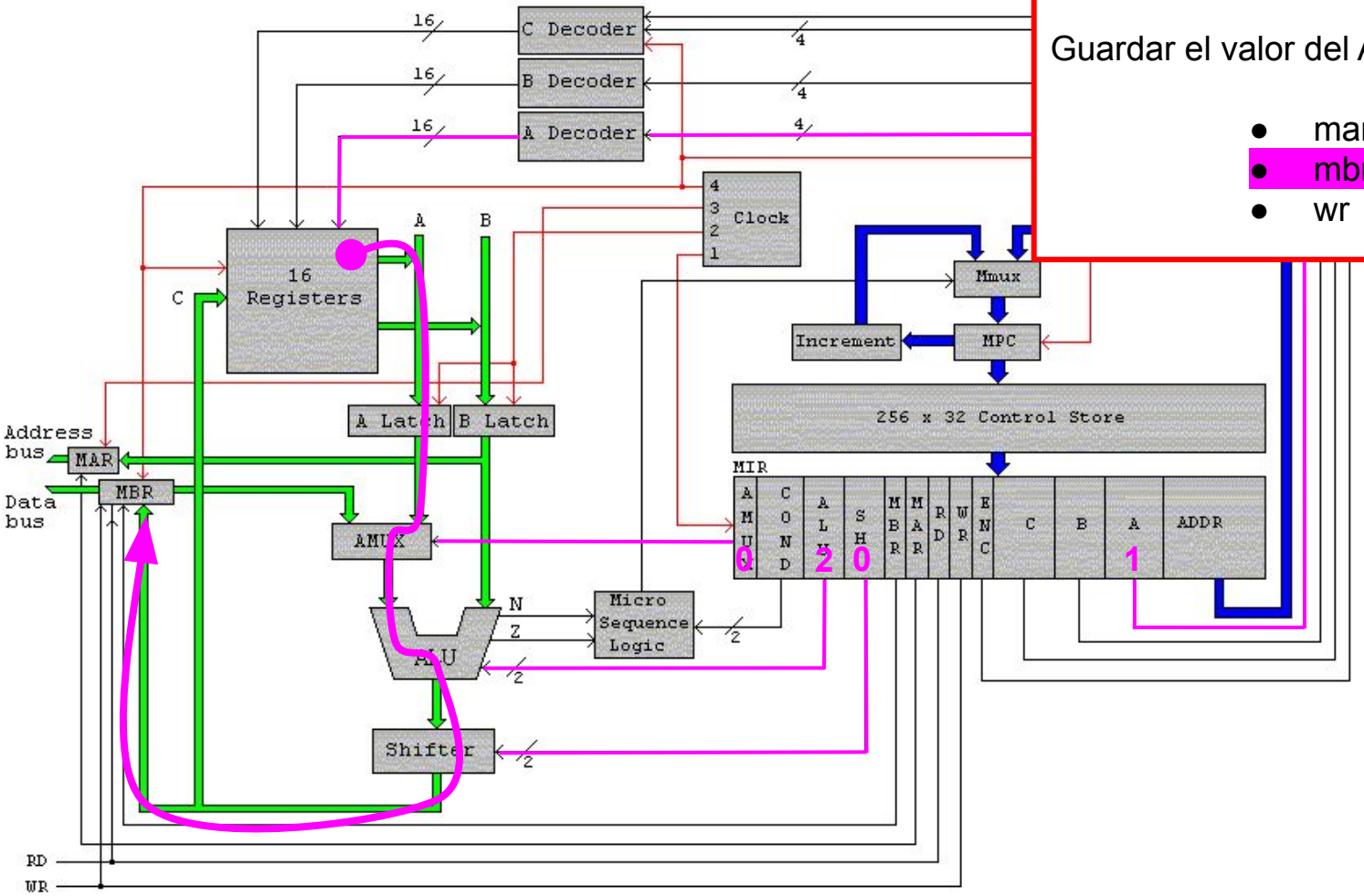
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



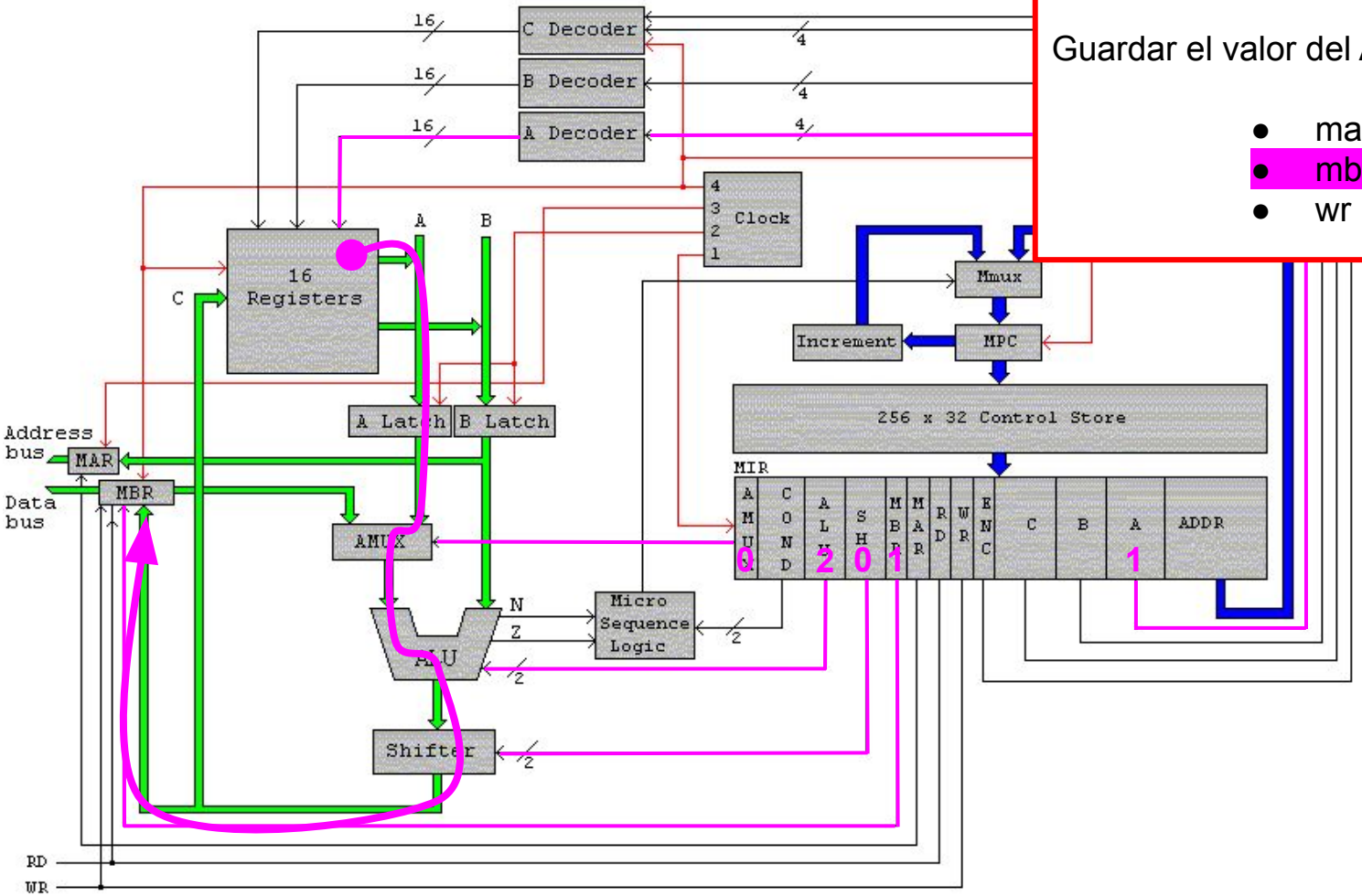
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

```
. . .
STOD   9
. . .
```

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



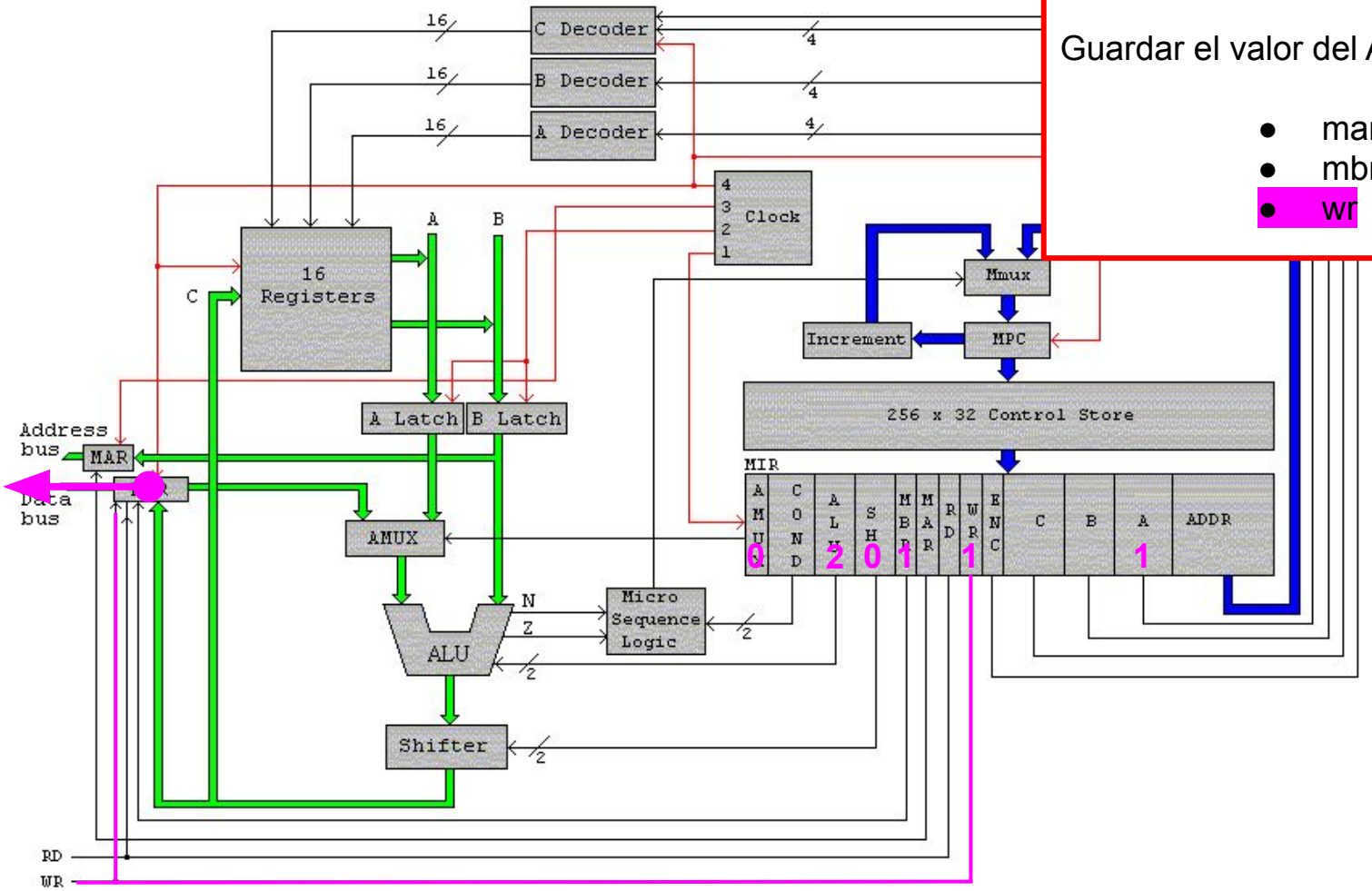
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



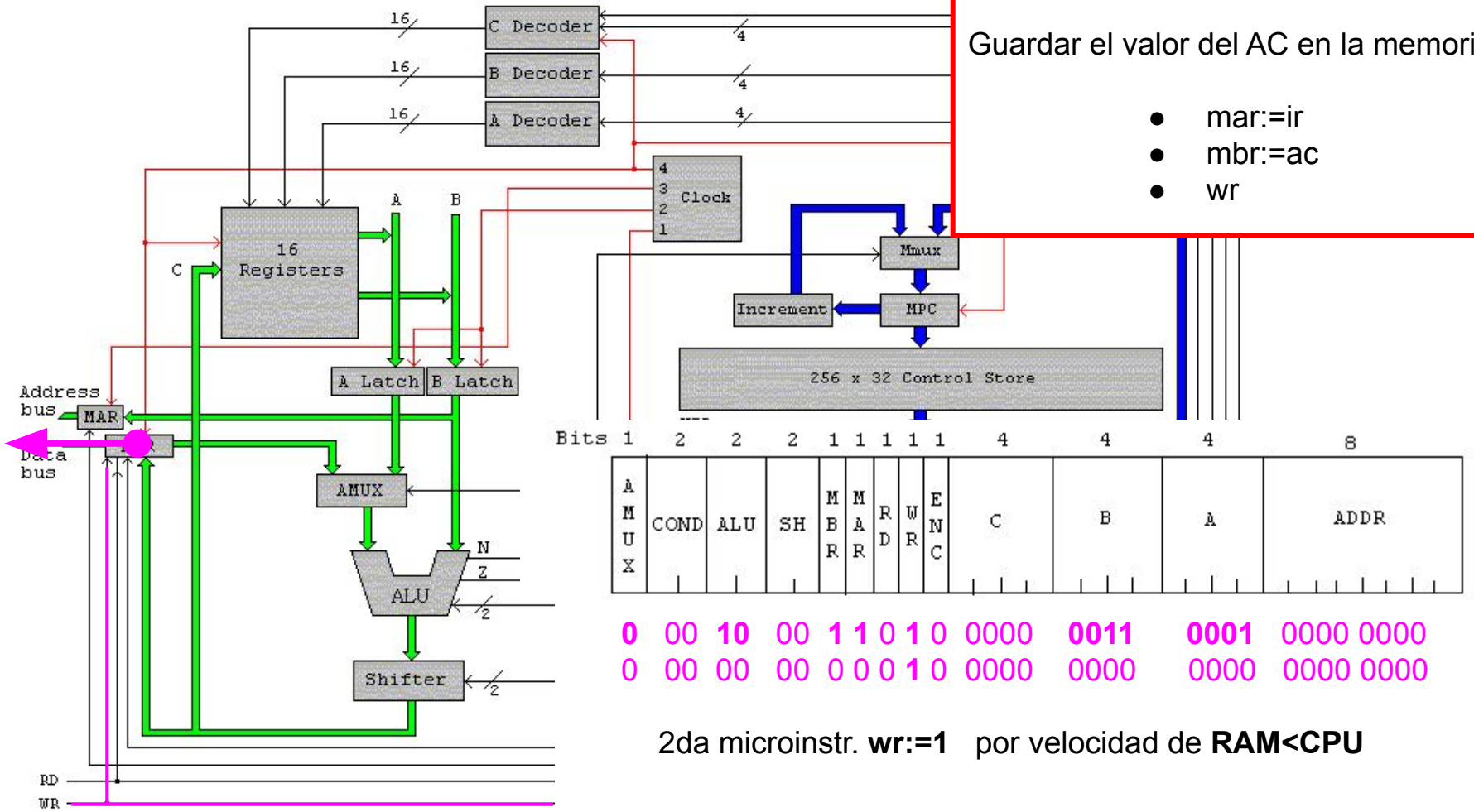
Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x

STORE DIRECT

. . .
STOD 9
. . .

Guardar el valor del AC en la memoria(9)

- mar:=ir
- mbr:=ac
- wr



2da microinstr. **wr:=1** por velocidad de **RAM<CPU**

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

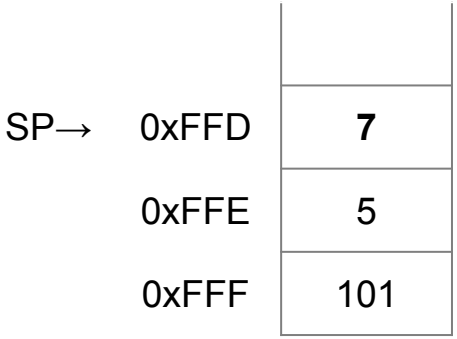
Pila o Stack

- La pila se ubica en la parte más alta de la memoria
- SP apunta siempre al tope de la pila
- POP saca el valor del tope de la pila
- PUSH pone un valor en la pila

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

Pila o Stack

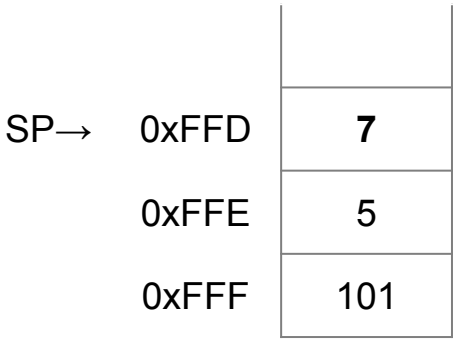
- La pila se ubica en la parte más alta de la memoria
- SP apunta siempre al tope de la pila
- POP saca el valor del tope de la pila
- PUSH pone un valor en la pila



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP

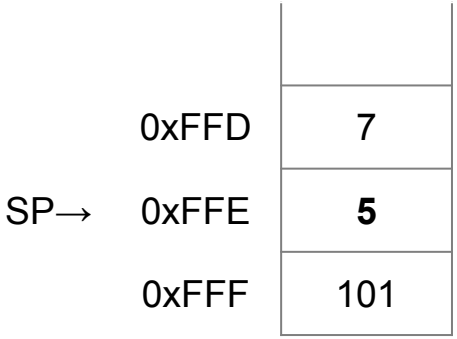
- SP ↔ 0xFFD
- m[sp] ↔ valor en el tope de la pila



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP

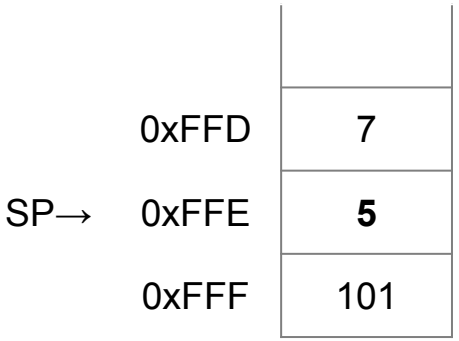
- SP ↔ 0xFFD
- m[sp] ↔ valor en el tope de la pila
- SP := SP+1 ↔ muevo el puntero



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

PUSH

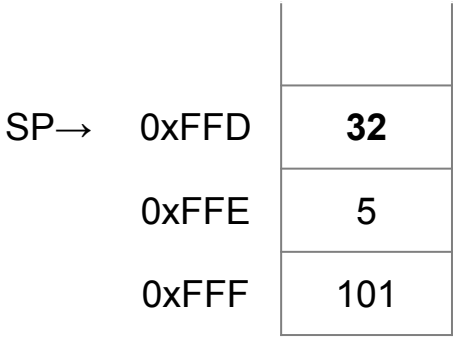
- SP ↔ 0xFFE



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
11111100yyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
11111110yyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

PUSH

- SP ↔ 0xFFE
- SP := SP-1 ↔ muevo el puntero
- m[sp] ←nuevo valor en la pila



Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP INDIRECT

. . .
POPI
. . .

Saca del tope de la pila y lo guarda en la memoria(ac)

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP INDIRECT

. . .
POPI
. . .

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

1111 0010 0000 0000

Binary	Mnemonic	Instruction	Meaning
0000xxxxxxxxxxxx	LODD	Load direct	ac:=m[x]
0001xxxxxxxxxxxx	STOD	Store direct	m[x]:=ac
0010xxxxxxxxxxxx	ADDD	Add direct	ac:=ac+m[x]
0011xxxxxxxxxxxx	SUBD	Subtract direct	ac:=ac-m[x]
0100xxxxxxxxxxxx	JPOS	Jump positive	if ac=>0 then pc:=x
0101xxxxxxxxxxxx	JZER	Jump zero	if ac=0 then pc:=x
0110xxxxxxxxxxxx	JUMP	Jump	pc:=x
0111xxxxxxxxxxxx	LOCO	Load constant	ac:=x (0<x<4095)
1000xxxxxxxxxxxx	LODL	Load local	ac:=m[sp+x]
1001xxxxxxxxxxxx	STOL	Store local	m[x+sp]:=ac
1010xxxxxxxxxxxx	ADDL	Add local	ac:= c+m[sp+x]
1011xxxxxxxxxxxx	SUBL	Subtract local	ac:=ac-m[sp+x]
1100xxxxxxxxxxxx	JNEG	Jump negative	if ac<0 then pc:=x
1101xxxxxxxxxxxx	JNZE	Jump nonzero	if ac<>0 then pc:=x
1110xxxxxxxxxxxx	CALL	Call procedure	sp:=sp-1; m[sp]:=pc; pc:=x
1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac
1111011000000000	POP	Pop from stack	ac:=m[sp]; sp:=sp+ 1
1111100000000000	RETN	Return	pc:=m[sp]; sp:=sp+ 1
1111101000000000	SWAP	Swap ac sp	tmp:=ac; ac:=sp; sp:=tmp
1111110yyyyyyyyy	INSP	Increment sp	sp:=sp+y (0<=y<=255)
1111111yyyyyyyyy	DESP	Decrement sp	sp:=sp-y (0<=y<=255)

POP INDIRECT

. . .
POPI
. . .

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu

1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

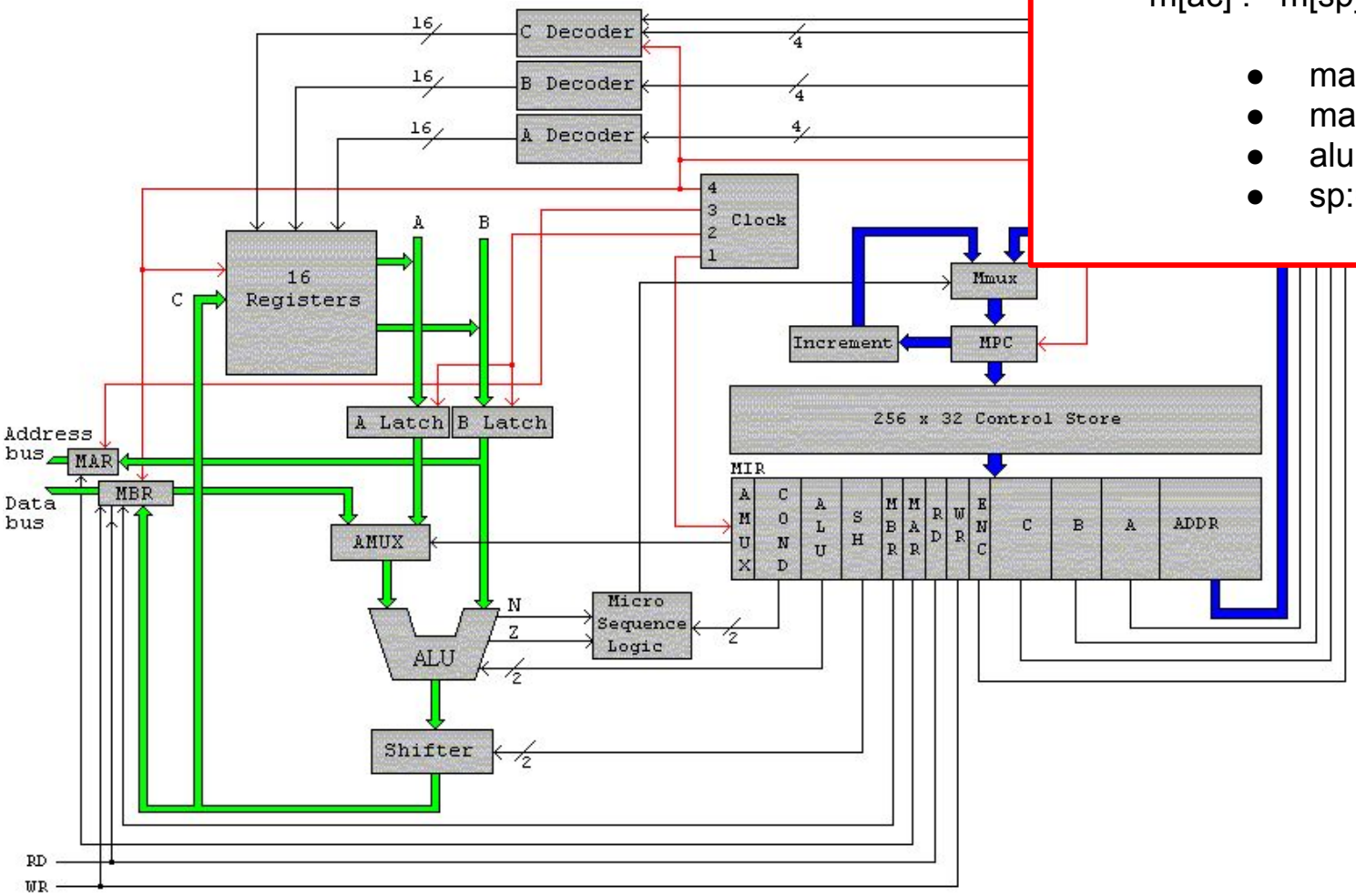
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

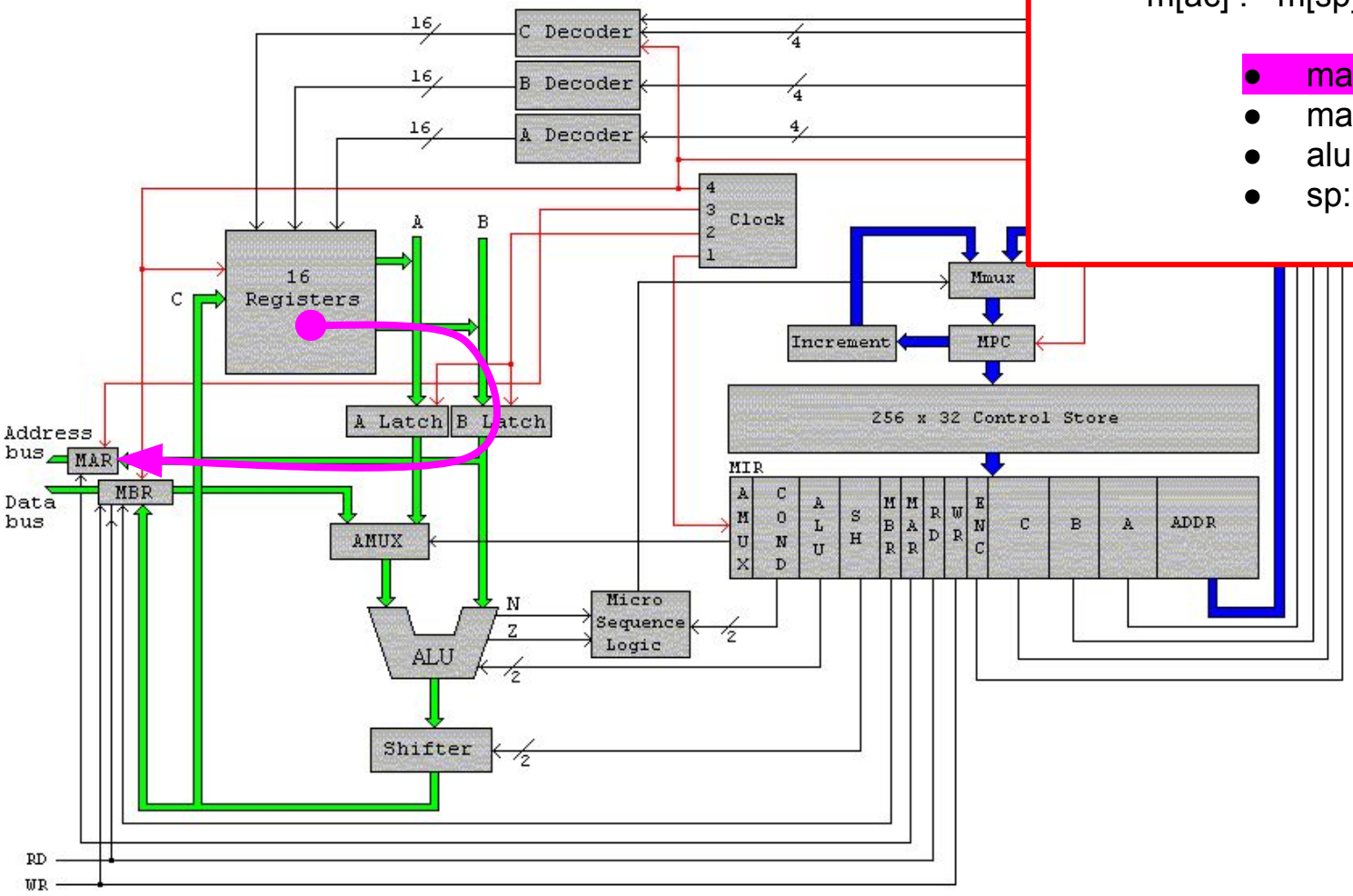
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

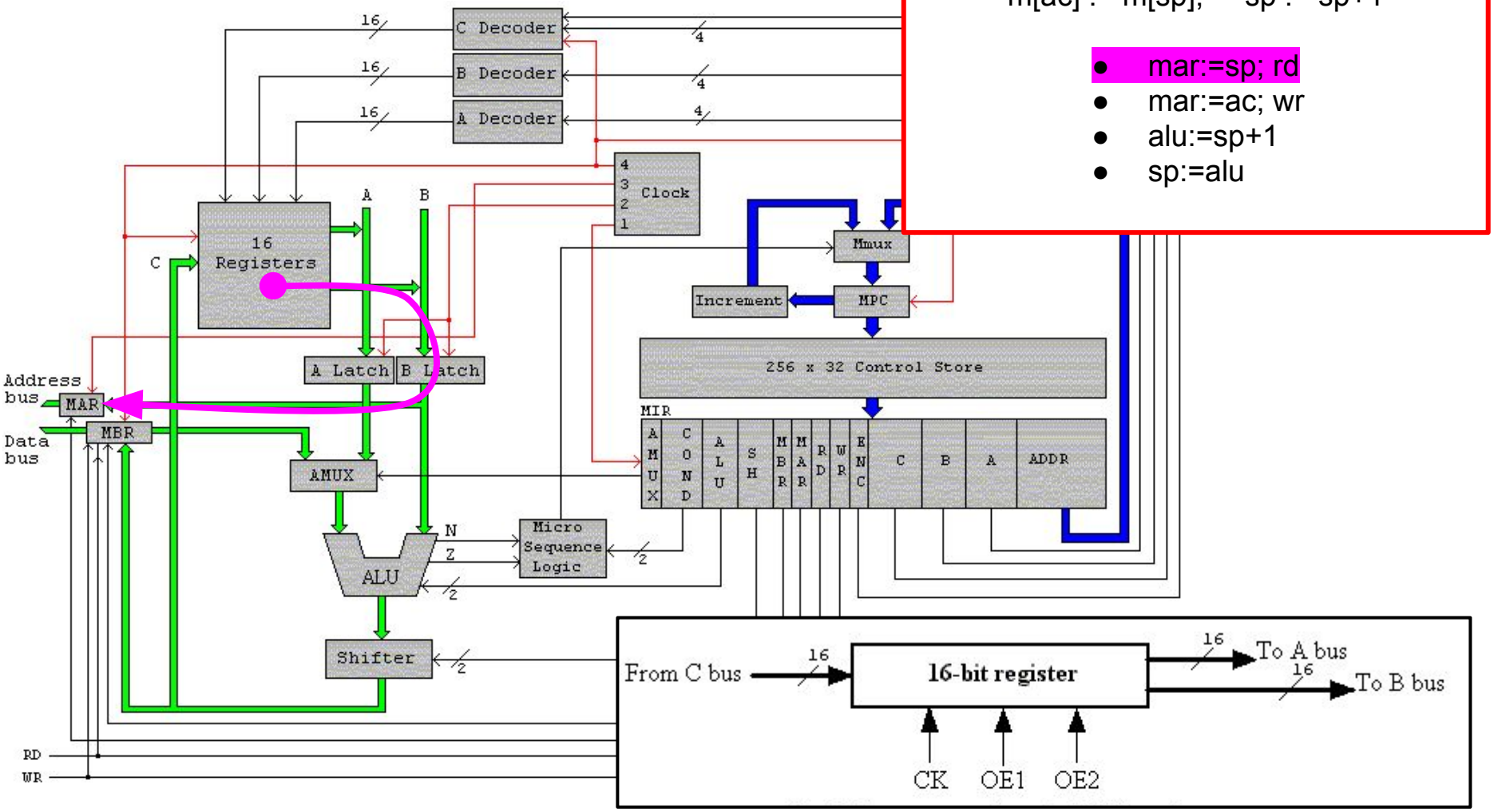
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

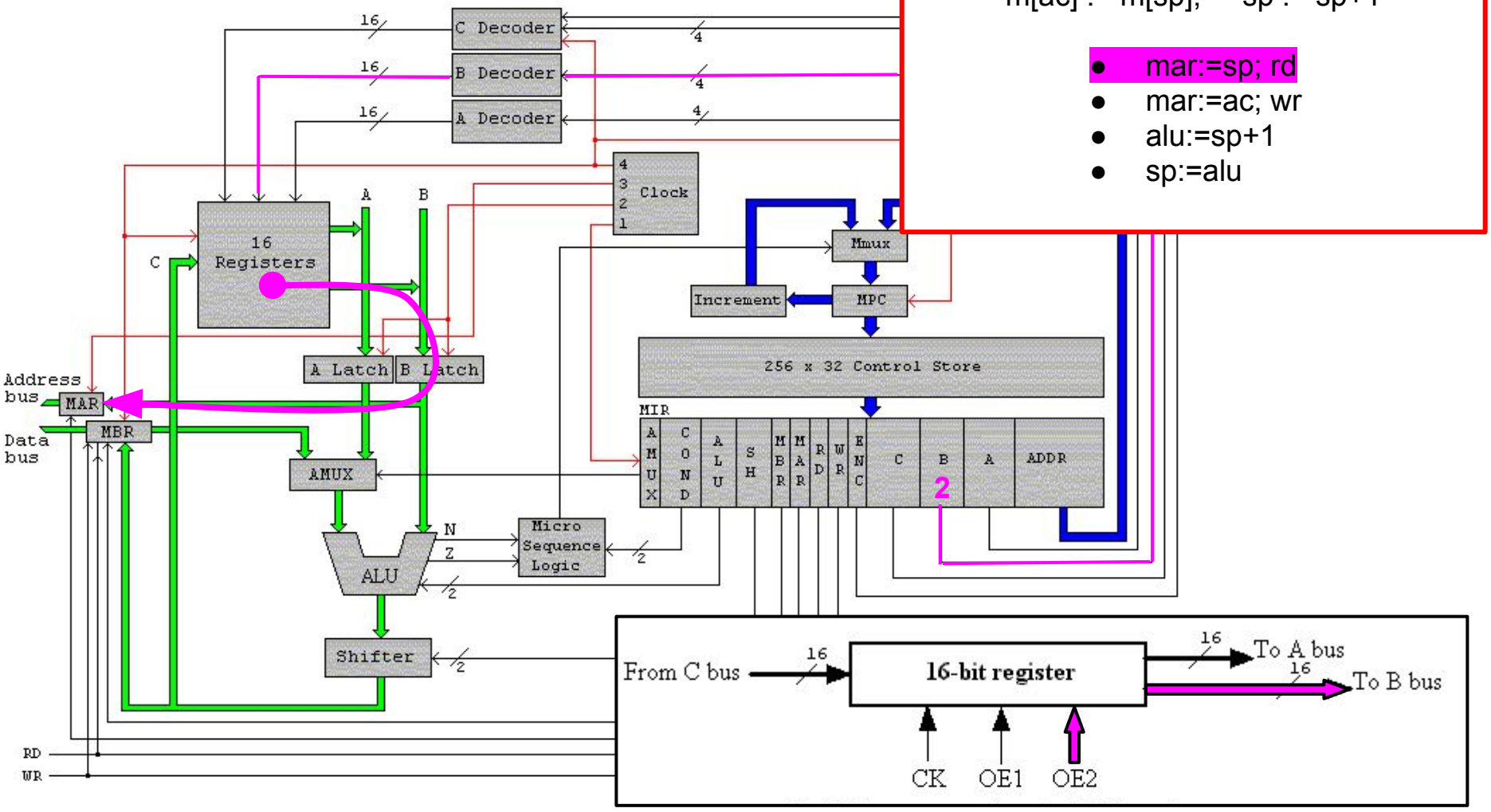
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

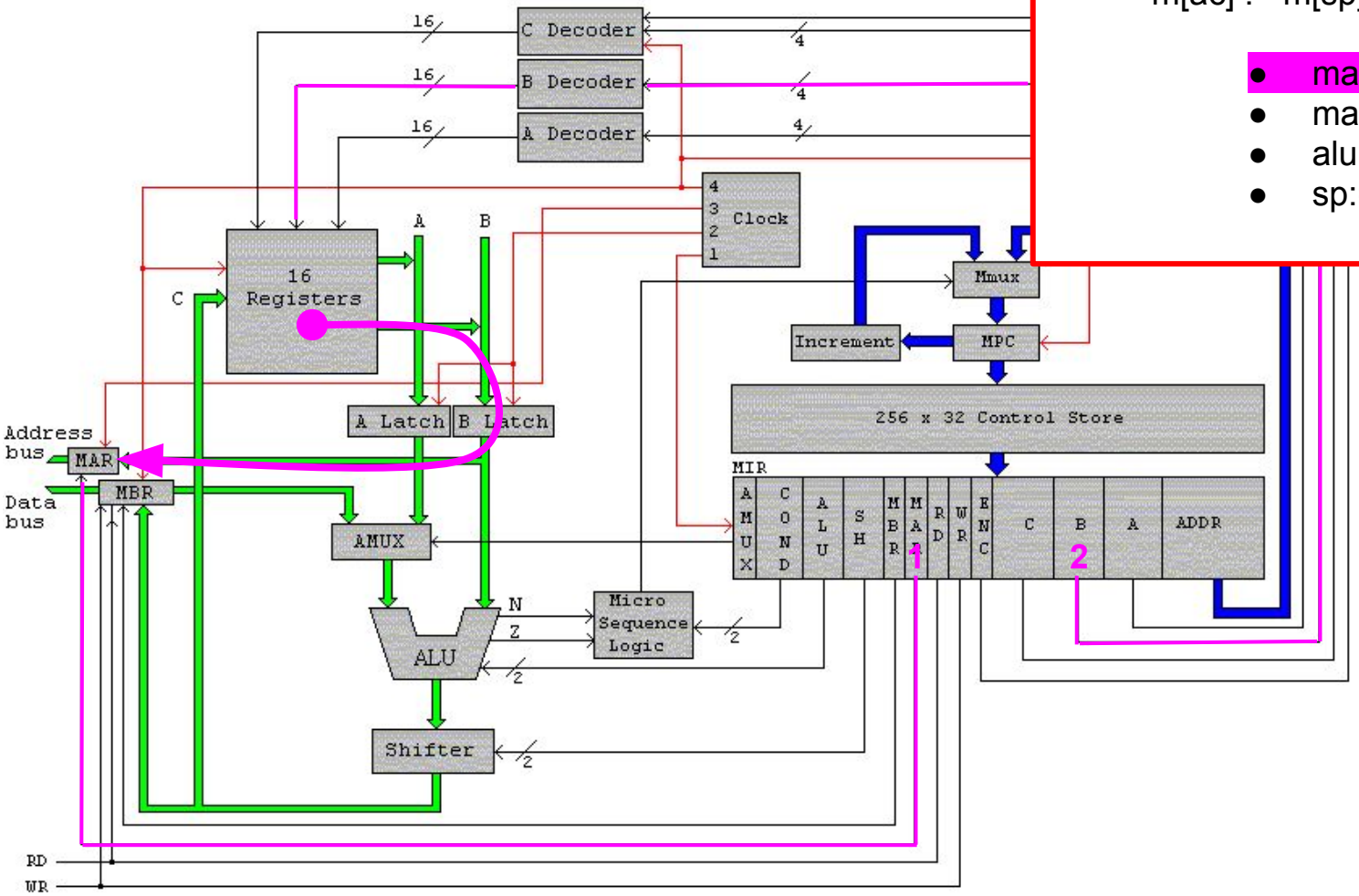
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

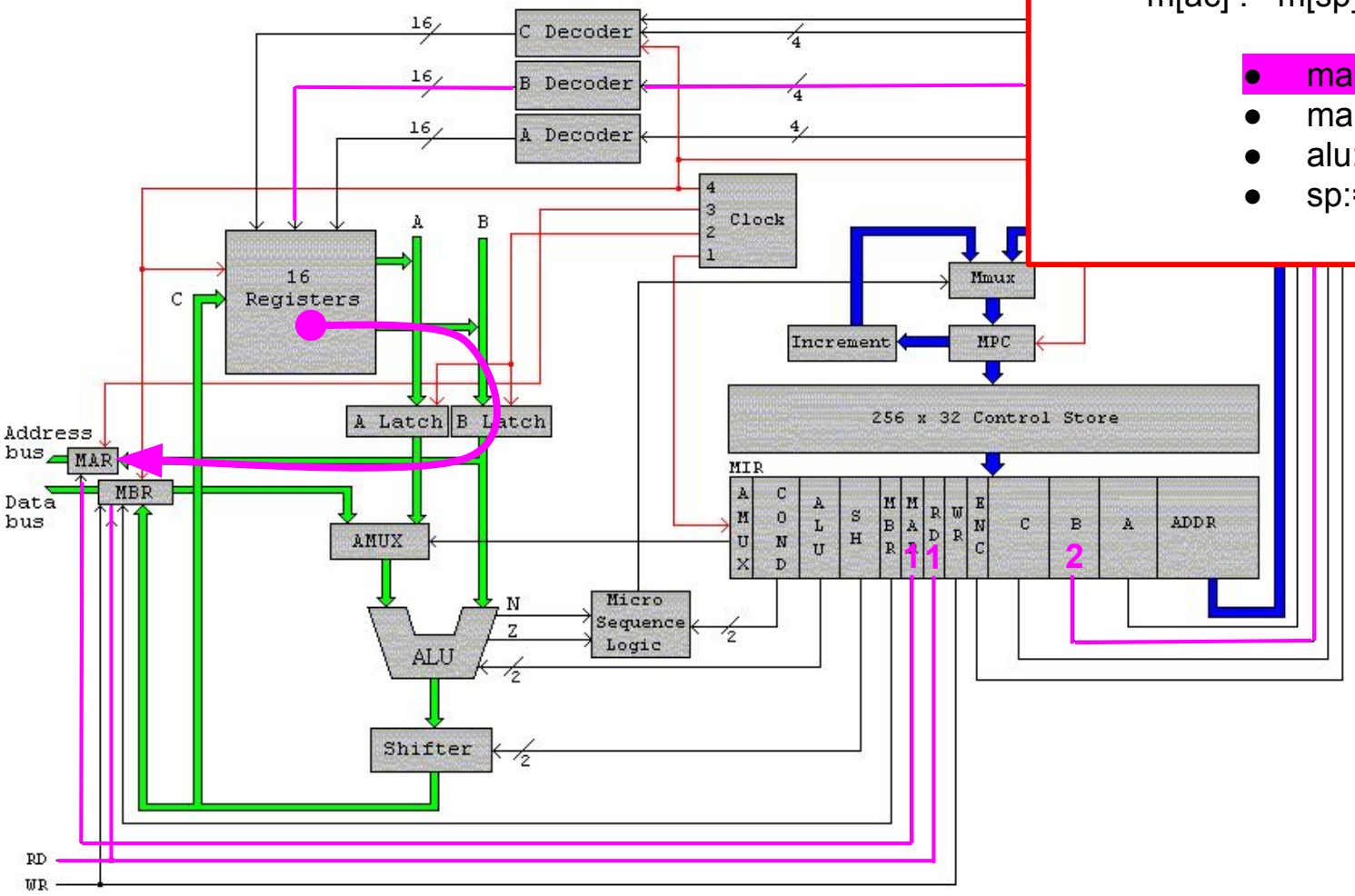
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

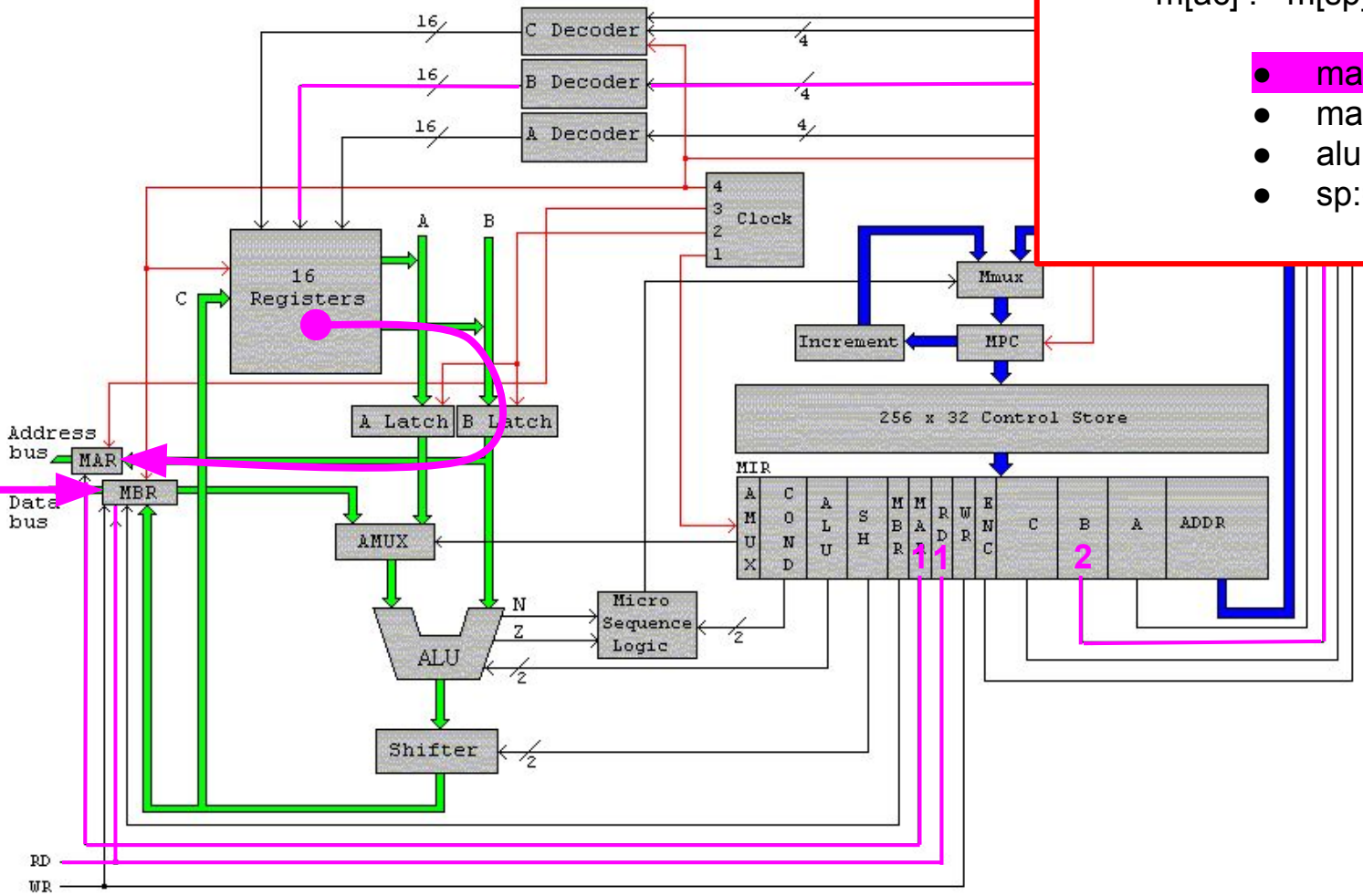
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

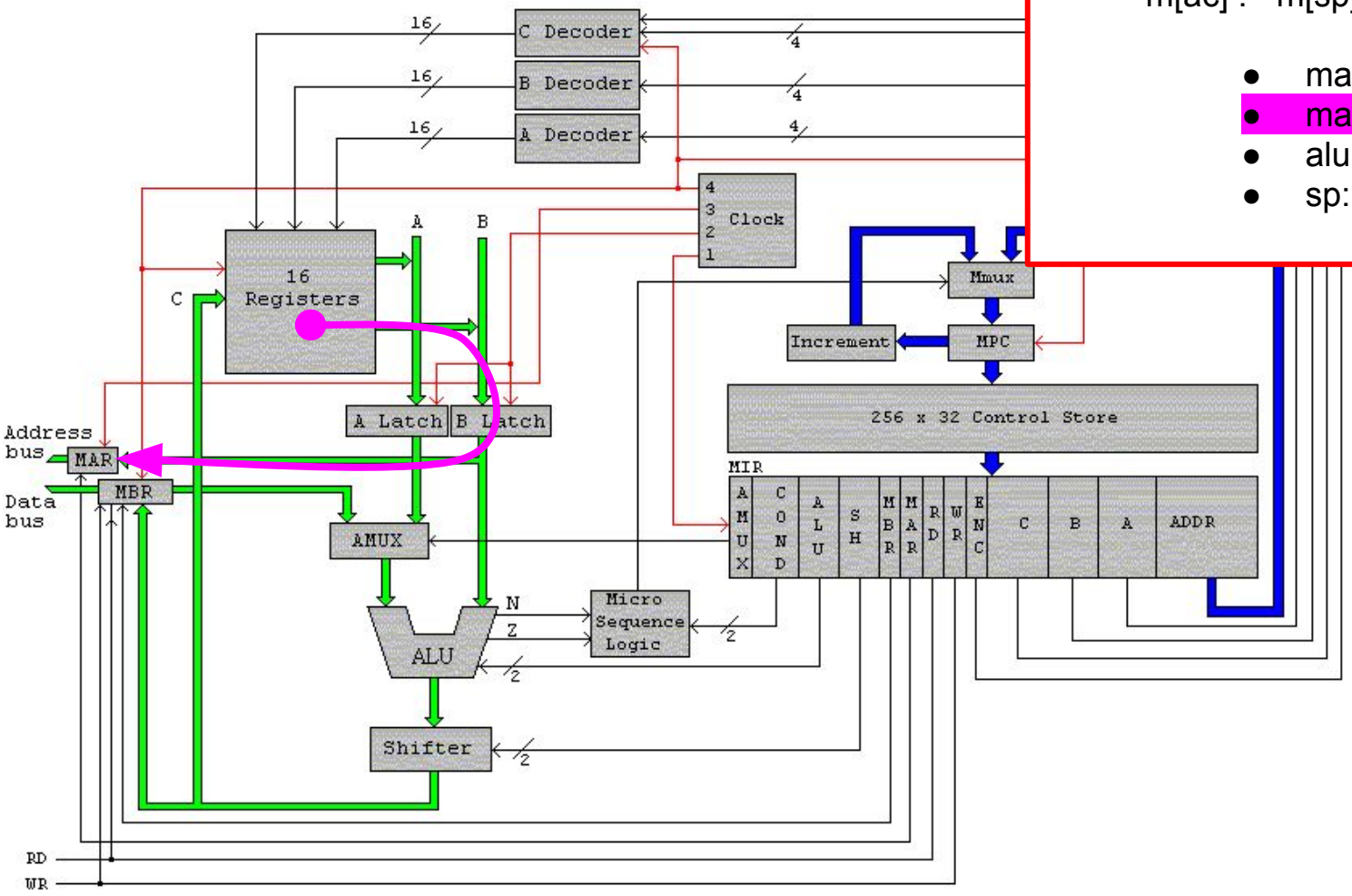
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

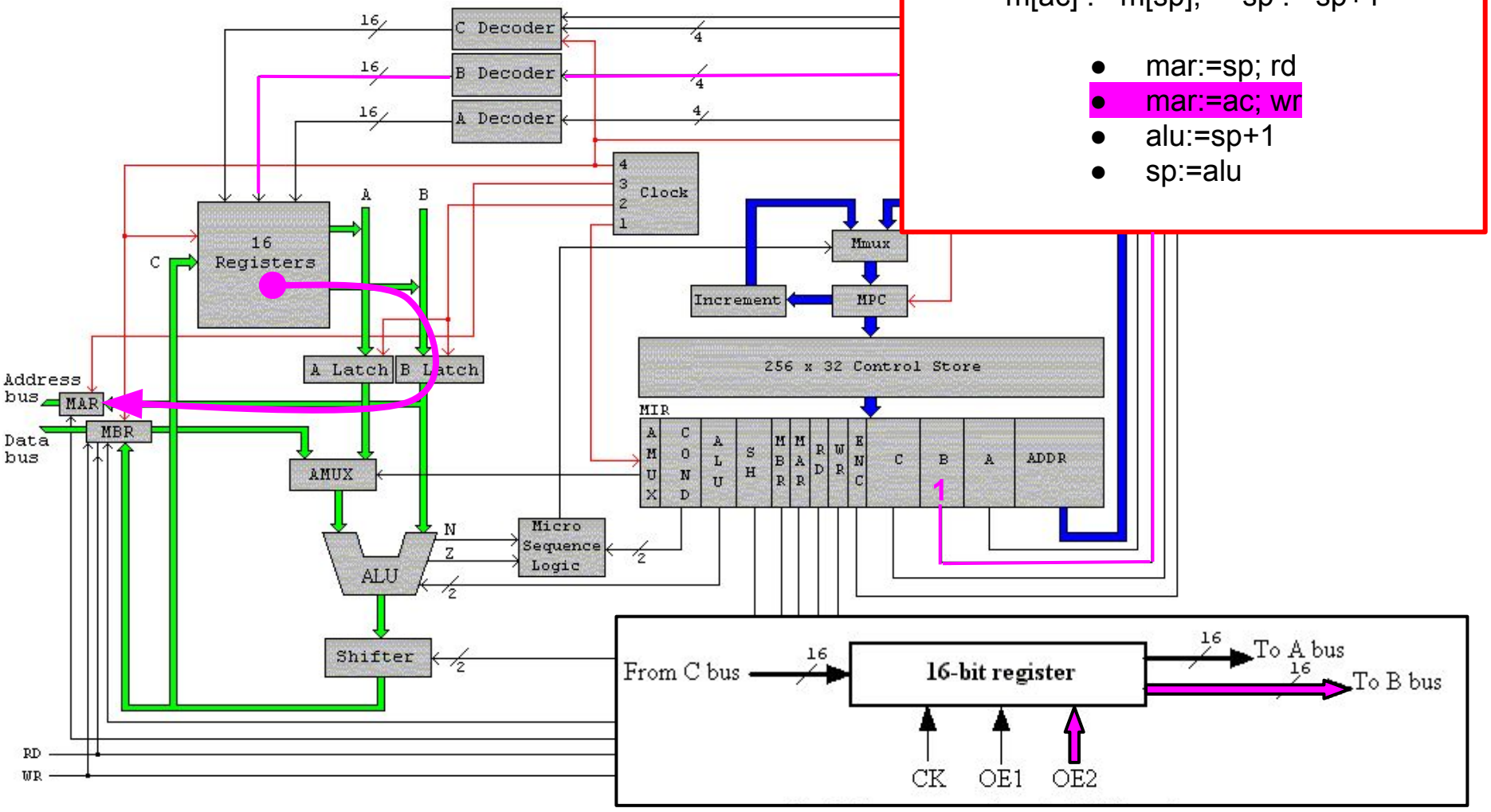
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

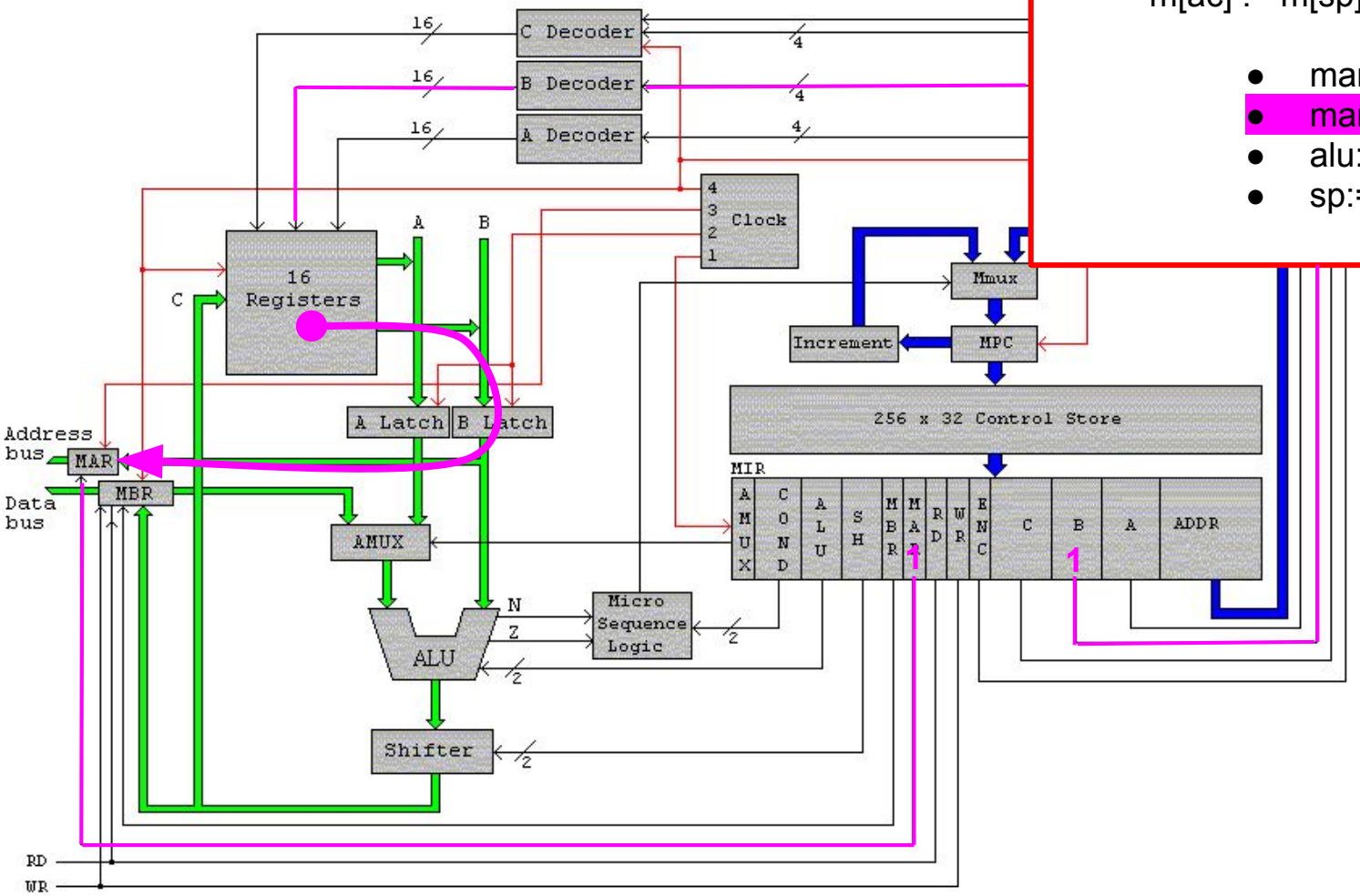
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

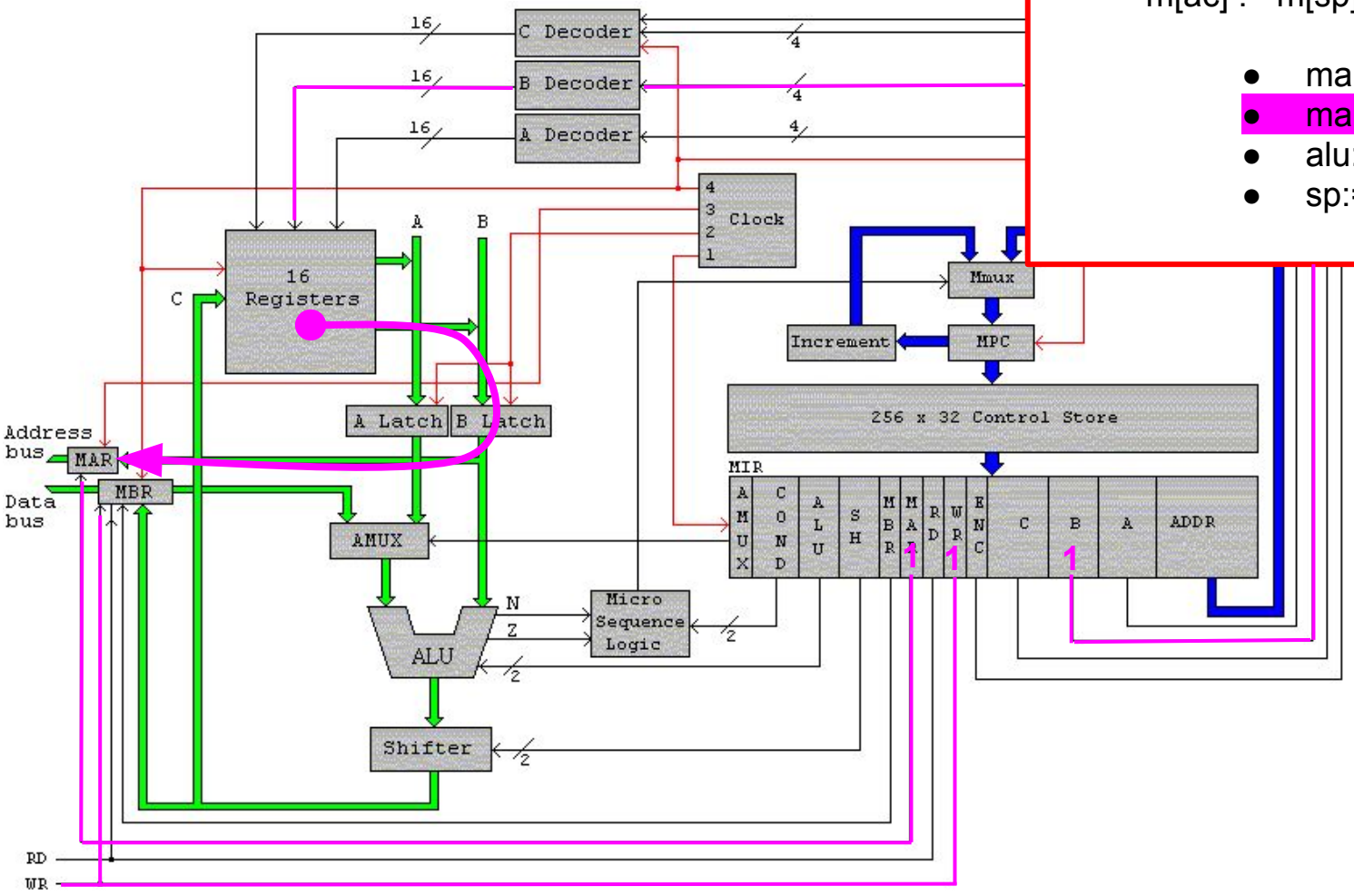
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

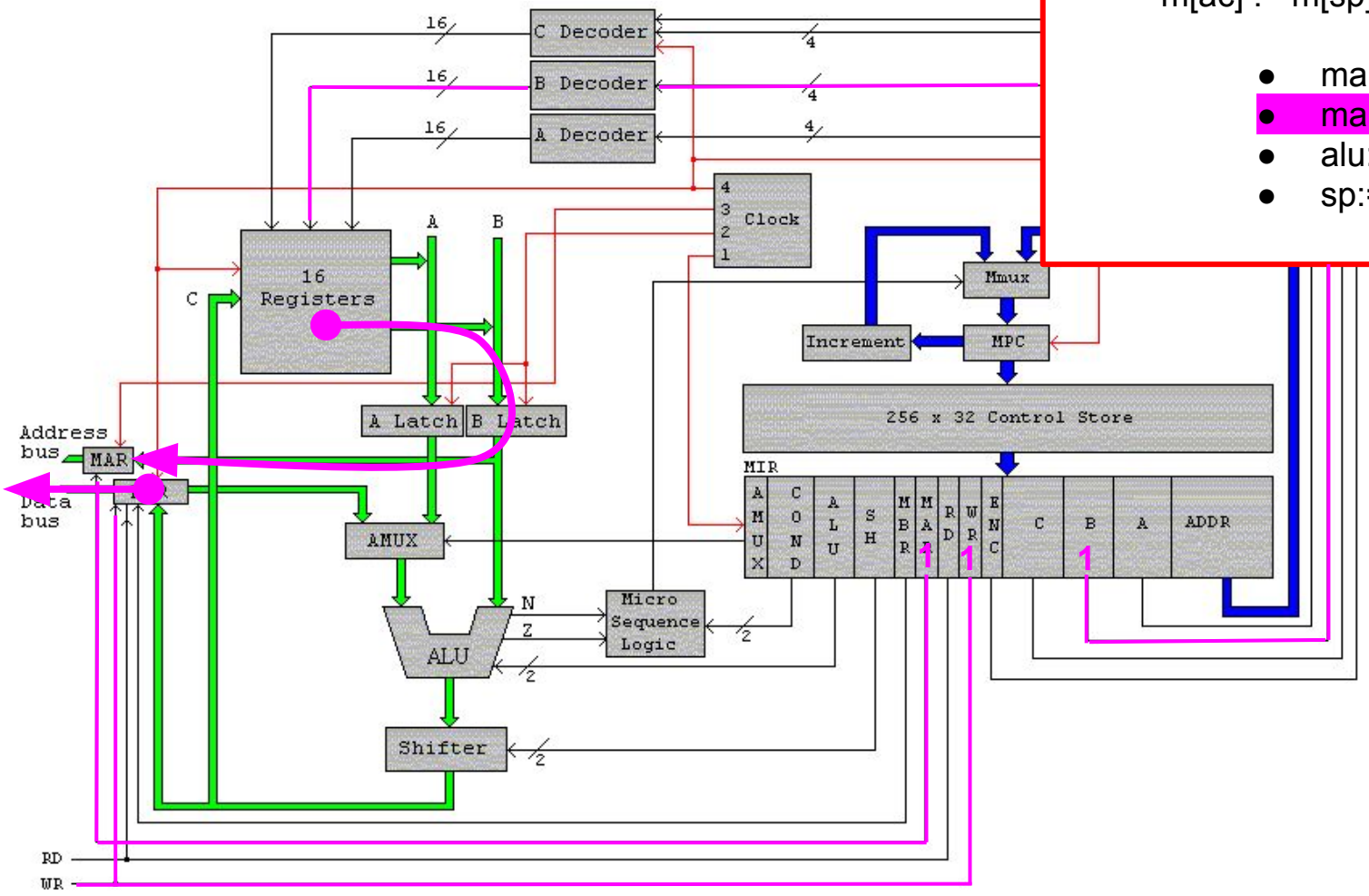
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

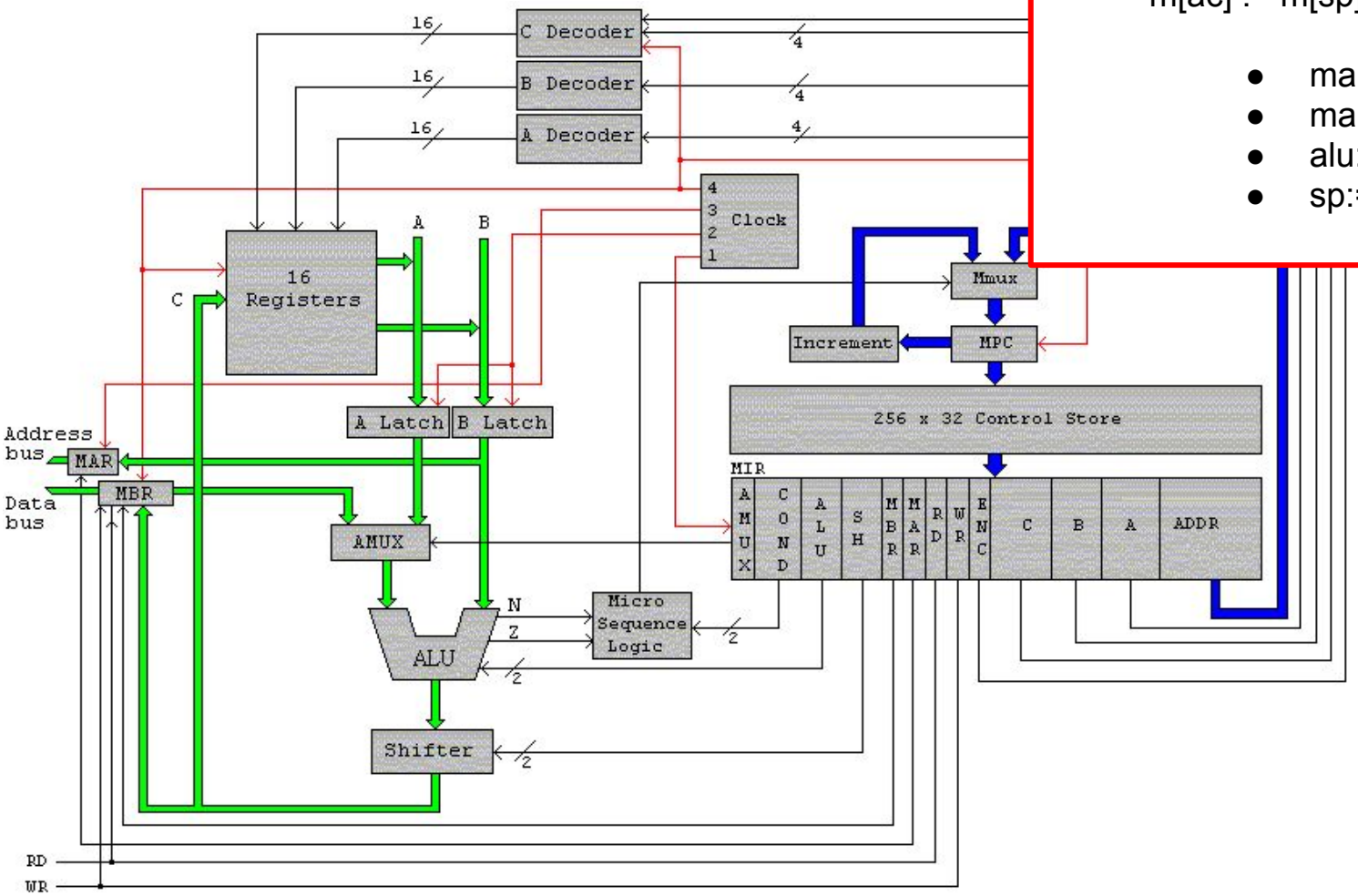
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

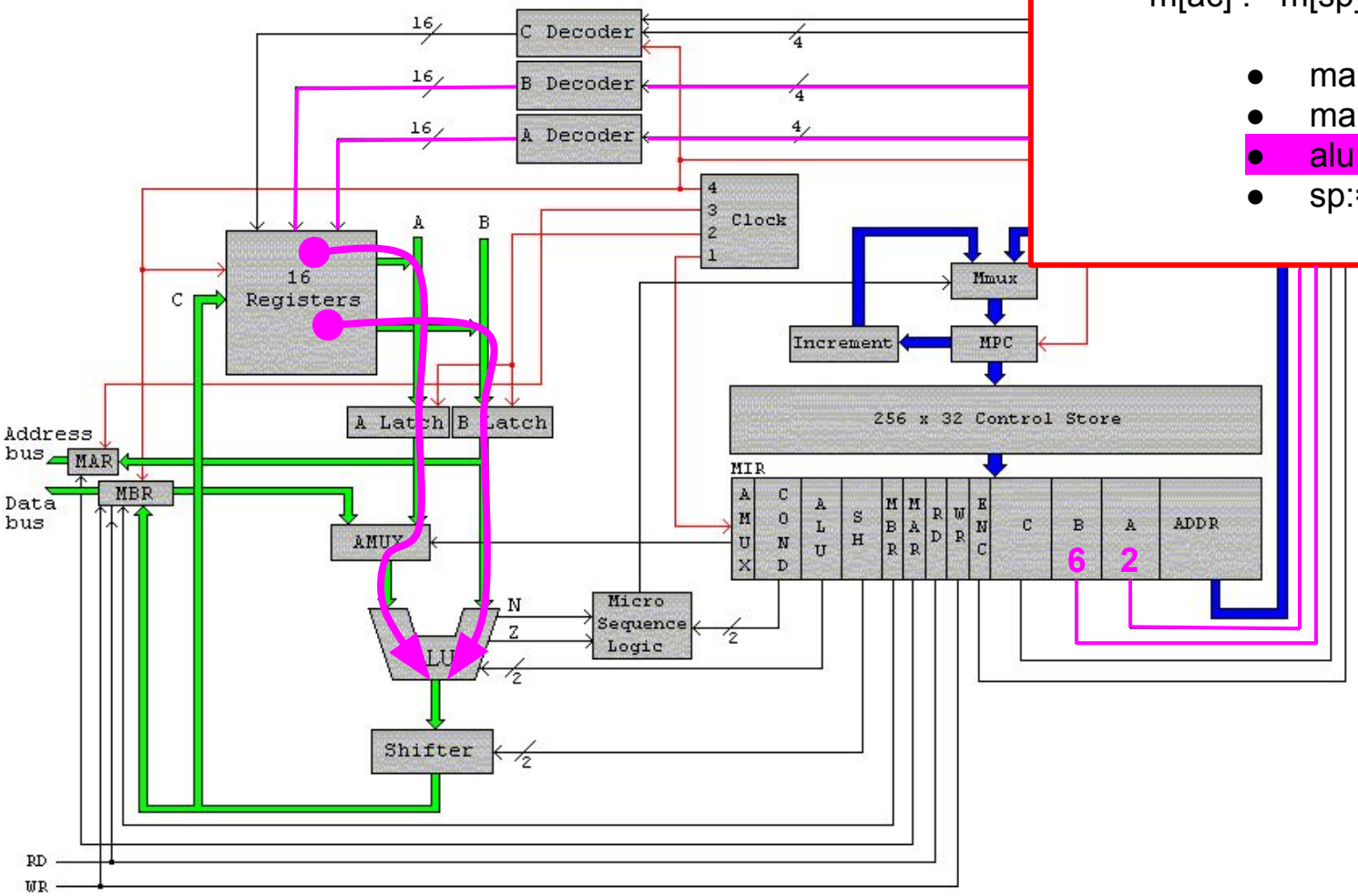
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

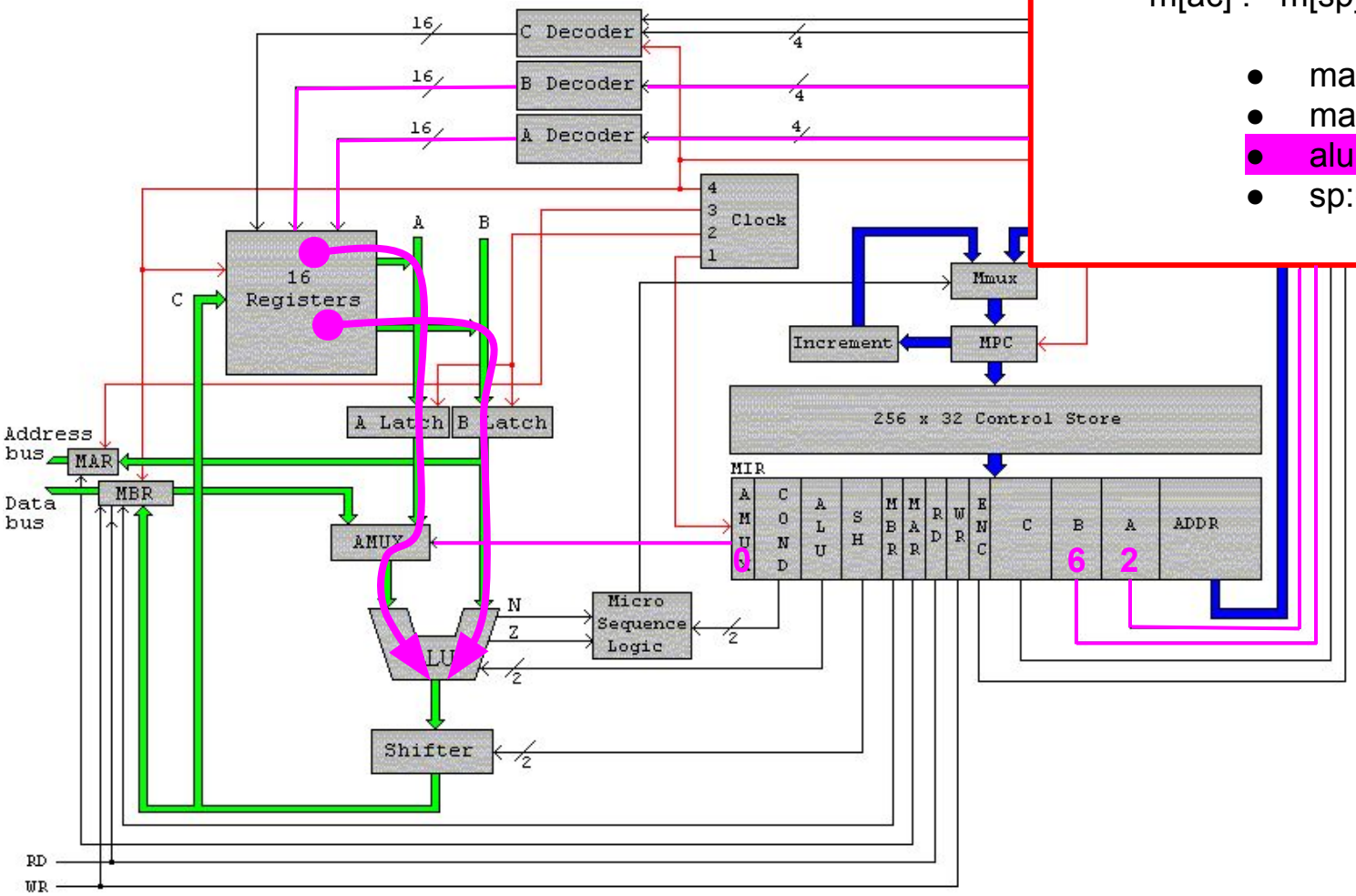
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

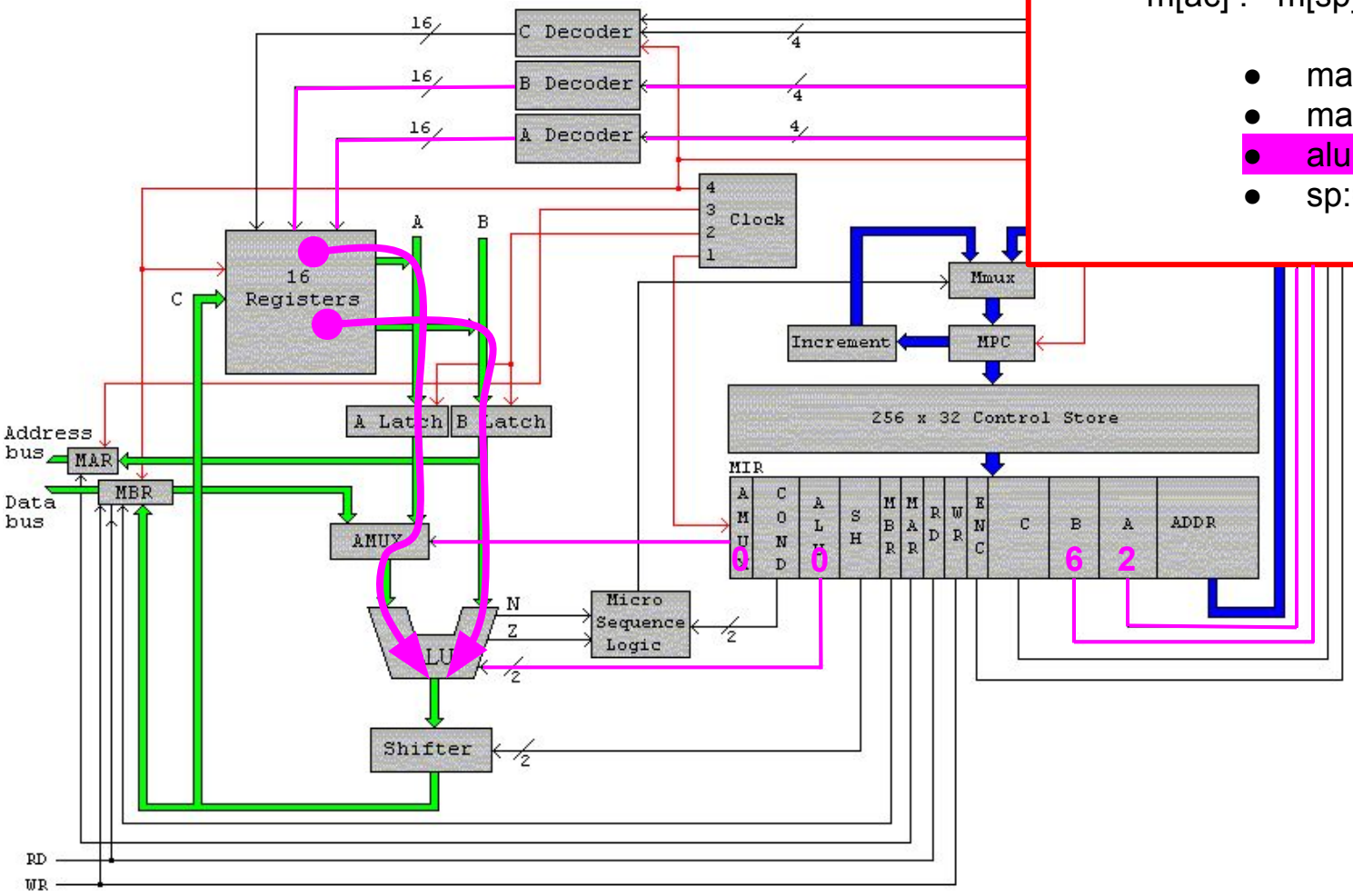
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

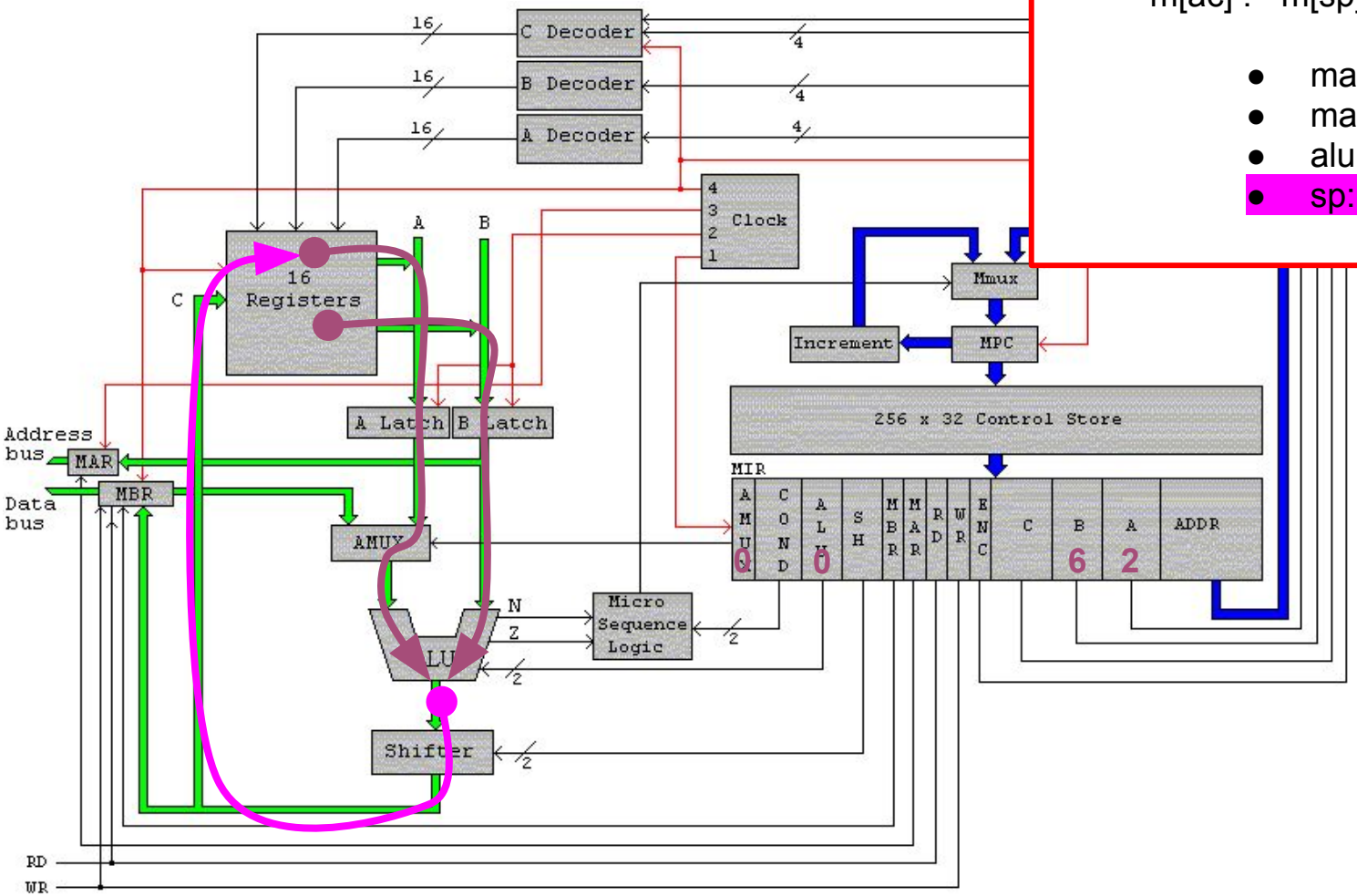
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

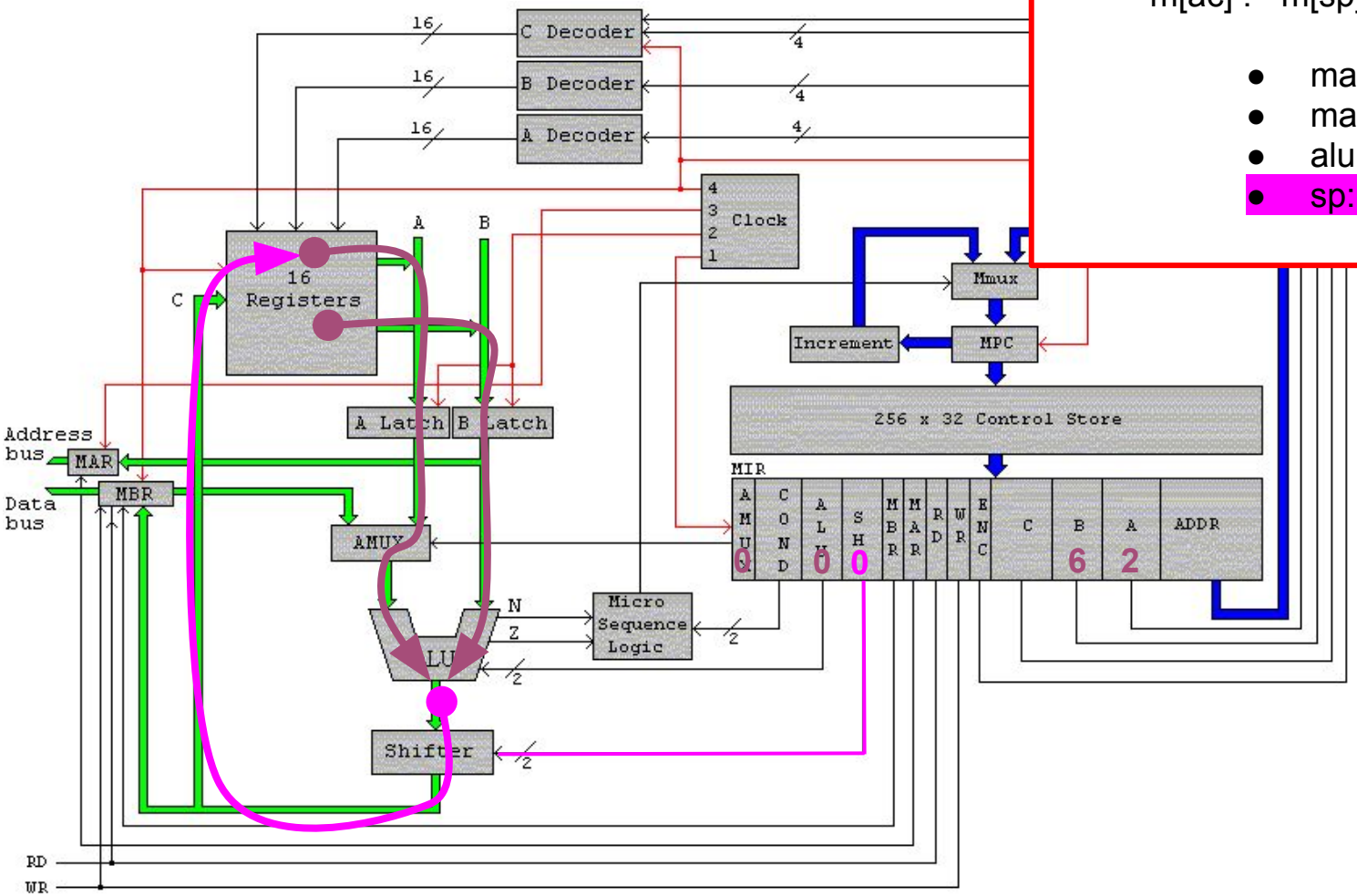
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

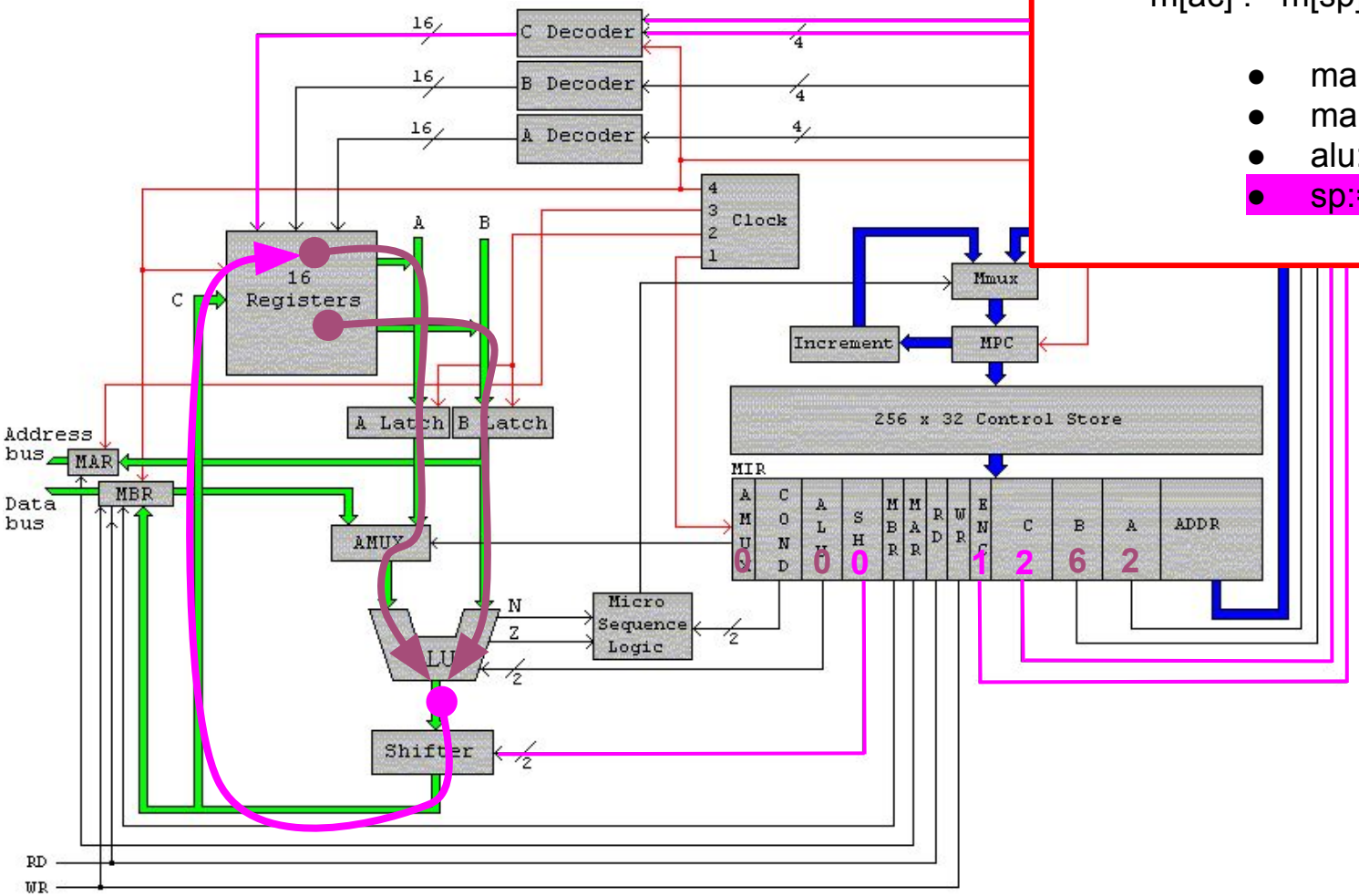
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

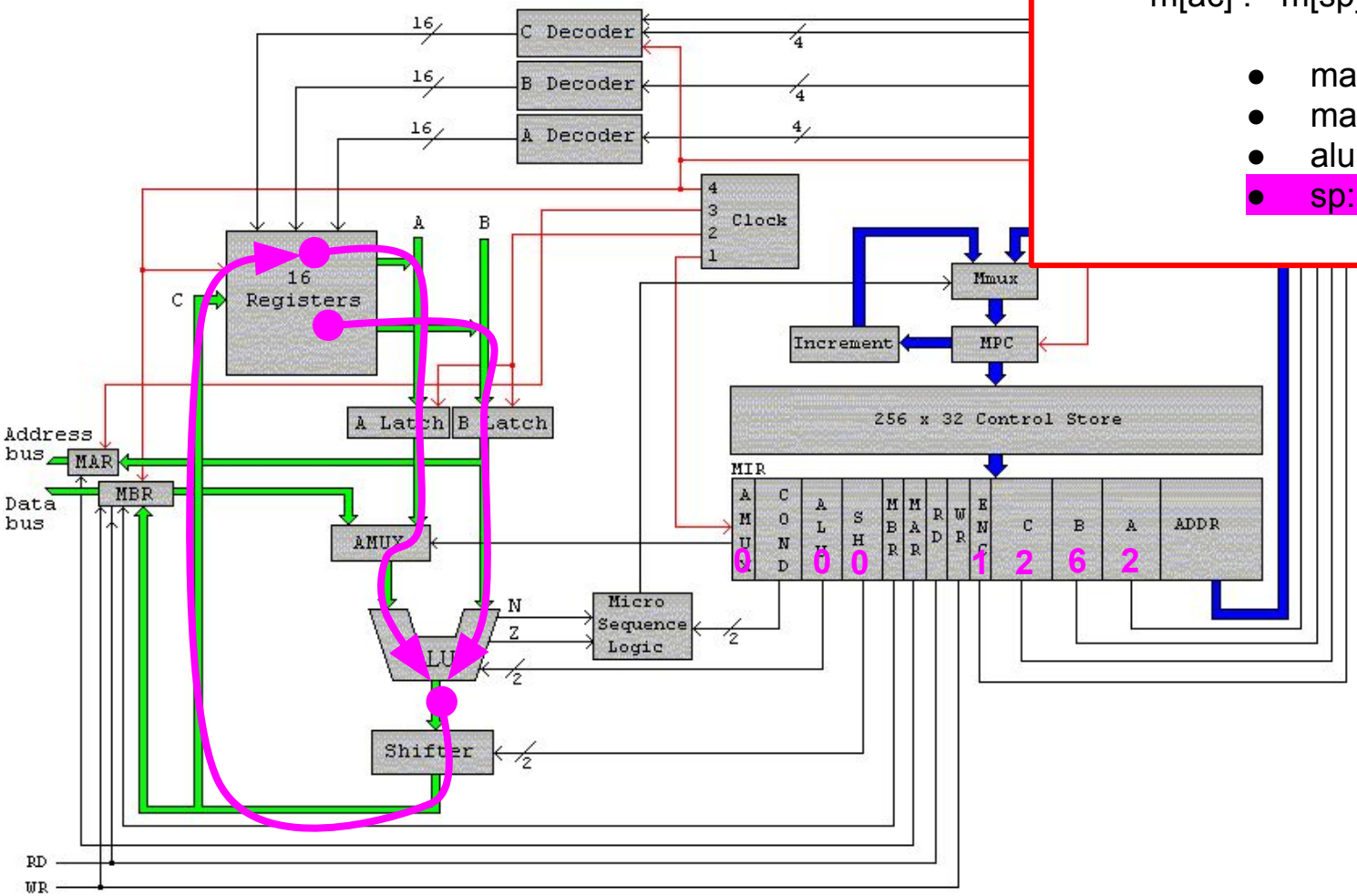
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



1111000000000000	PSHI	Push indirect	sp:=sp-1; m[sp]:=m[ac]
1111001000000000	POPI	Pop indirect	m[ac]:=m[sp]; sp:=sp+1
1111010000000000	PUSH	Push onto stack	sp:=sp-1; m[sp]:=ac

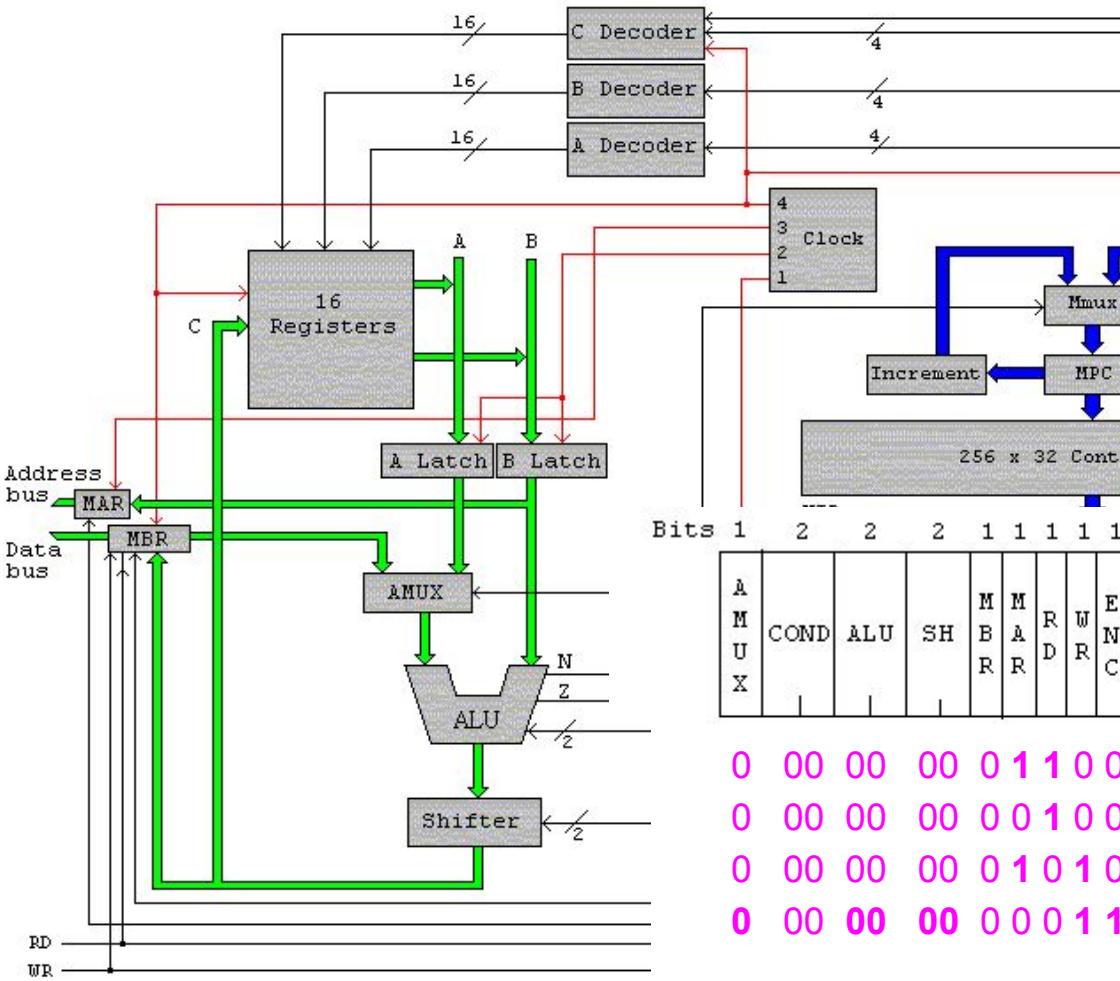
POP INDIRECT

POPI

Saca del tope de la pila y lo guarda en la memoria(ac)

m[ac] := m[sp]; sp := sp+1

- mar:=sp; rd
- mar:=ac; wr
- alu:=sp+1
- sp:=alu



Bits	1	2	2	2	1	1	1	1	1	4	4	4	8
	AMUX	COND	ALU	SH	MBR	MAR	RD	WR	ENC	C	B	A	ADDR
	0	00	00	00	0	**1**	**1**	0	0	0000	**0010**	0000	0000 0000
	0	00	00	00	0	0	**1**	0	0	0000	0000	0000	0000 0000
	0	00	00	00	0	**1**	0	**1**	0	0000	**0001**	0000	0000 0000
	0	00	**00**	**00**	0	0	0	**1**	**1**	**0010**	**0110**	**0010**	0000 0000

Resumiendo

- Una instrucción de C o Pascal se descompone en varias instrucciones assembler
- Una instrucción assembler se descompone en una o varias acciones que debe realizar el micro-procesador
- Una acción se resuelve en una o varias micro-instrucciones
- Cada micro-instrucción consume 1 ciclo del clock

Arquitectura Horizontal

T1 - Trayectoria de ejecución

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta